

Dataset

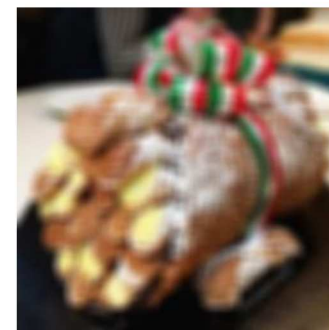
- Tratto da FoodX-251^[1]
- 251 classi di immagini cibo
- Training:
 - 5020 immagini etichettate (20 per classe)
 - 113455 immagini non etichettate
- Test:
 - 11994 immagini etichettate "acquisite in condizioni ideali"
 - 11994 immagini etichettate "acquisite in condizioni non ideali"

[1] : Kaur, Parneet, et al. "Foodx-251: a dataset for fine-grained food classification." arXiv preprint arXiv:1907.06167 (2019).

versione non-degraded



versione degraded



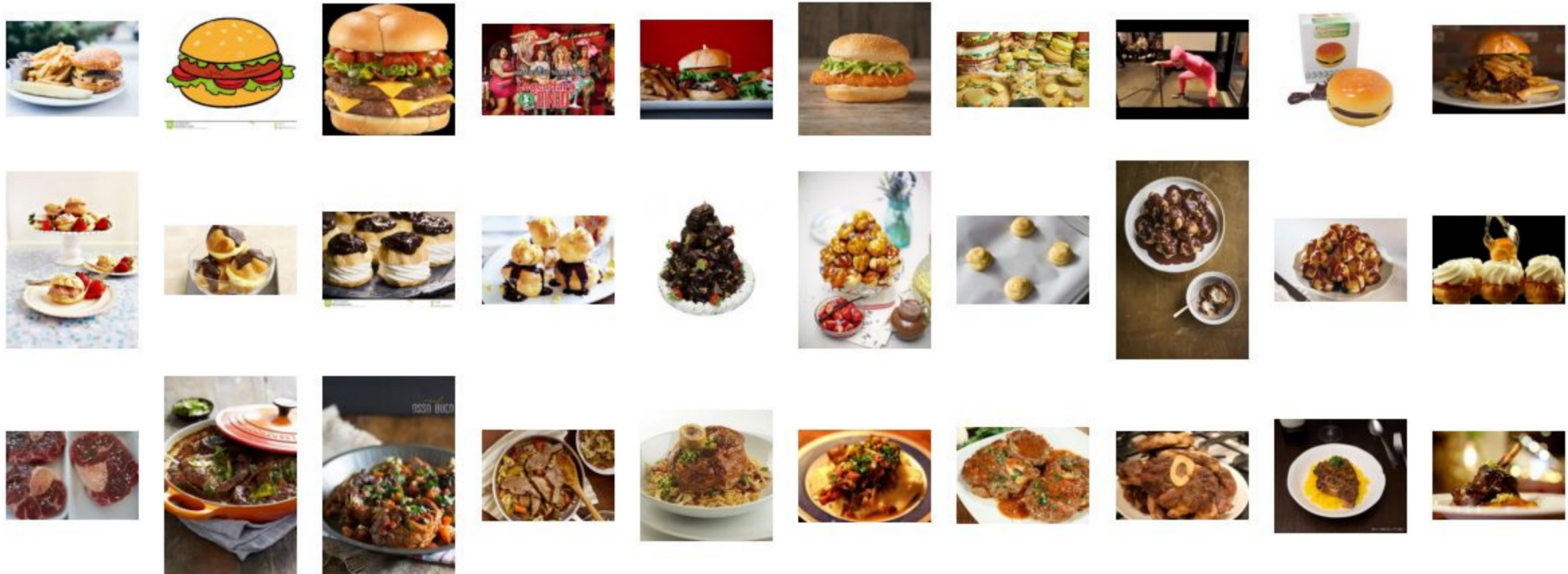
Sfide del Dataset e del dominio:

- Poche immagini di training etichettate
 - 20 immagini per classe
- Dominio complesso
 - Elevata variabilità intragruppo (oggetti non rigidi, sfondi differenti...)
 - Scarsa variabilità intergruppo
 - non sempre solo un piatto nelle foto
- Dati di bassa qualità, noisy labels
 - Le immagini a destra rappresentano tutte “nachos”



Sfide del Dataset e del dominio:

Di seguito: hamburger, profiterole, osso buco



Sfide del Dataset e del dominio:



sono...di tre classi diverse!

Assunzioni, obiettivi e premesse

- Sviluppo diviso in 2 parti:
 - Sviluppo di un modello predittivo, valutando le performance su dati “puliti”
 - Valutazione e ottimizzazione delle performance su dati “sporchi”
- Risorse temporali e computazionali limitate
 - Sfruttati esclusivamente Personal Computer
 - Approccio “Proof of concept”
- Metriche per la valutazione:
 - Accuracy
 - Rapporto tra il numero di predizioni corrette ed il numero di predizioni totali
 - metrica standard per la classificazione
 - Top5 - Accuracy
 - Rapporto tra il numero di predizioni in cui la classe corretta era tra le prime 5 per confidenza ed il numero di predizioni totali
 - Adatta alle sfide del dominio

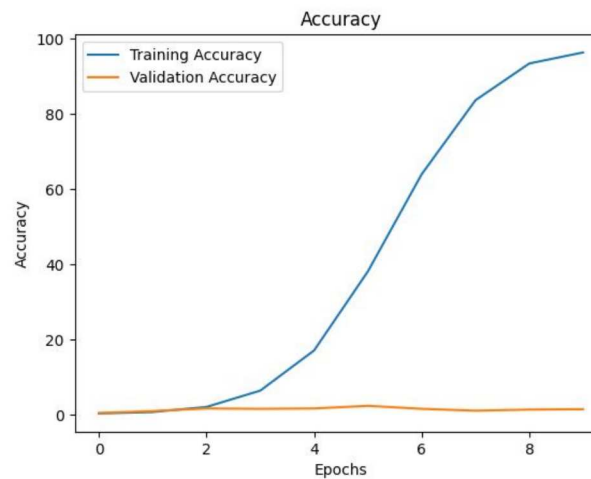
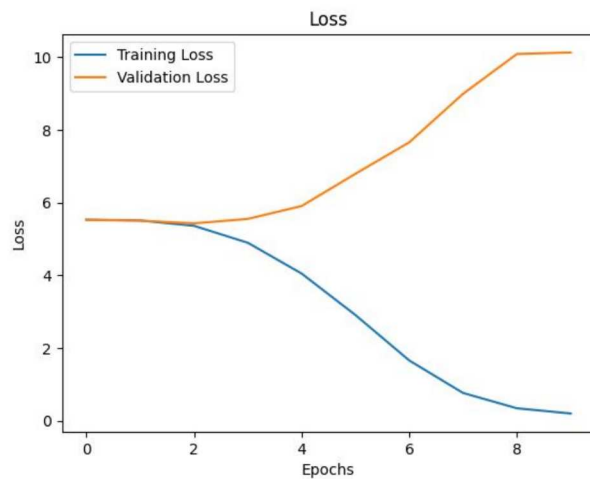
Approcci provati ma non approfonditi qui:

- Bag of Words (BoW):
 - metodo lento, poco testato, con risultati pessimi
- Ensemble di modelli:
 - Creare un ensemble di modelli che singolarmente sono buoni su un sottoinsieme del dominio
 - Ottimi risultati sui singoli modelli, ma non buoni quando uniti
- Augmentation dati di training
 - provato con il transfer learning di AlexNet
 - seppur promettente, dà risultati non migliori del caso con dati originali
- Allenamento dei modelli basati su transfer learning utilizzando k-fold

Step iniziali e baseline

- Esplorazione ed analisi del Dataset:
 - Calcolo dei parametri di normalizzazione
- Creazione di un validation set etichettato
 - Divisione del dataset di training etichettato in training e validation (80%-20%)
 - Classi bilanciate (16 immagini per classe nel training, 4 per classe nel validation)
- Definizione di un semplice modello baseline convoluzionale:
 - 3 blocchi convoluzionali (Conv->ReLU->MaxPool)
 - Dropout
 - 2 strati FC
- Addestrato sulle immagini di training etichettate

Risultati Baseline



Problemi:

- Overfitting

Possibili cause:

- Modello troppo complesso per il dominio (estremamente improbabile [1])
- Poche immagini, e "difficili", -> non è in grado di generalizzare

- 10 epoche, Cross Entropy, ADAM optimizer, learning rate 1e-4, 64 batch size
- Accuracy (validation): 1.59%
- Top5-Accuracy (validation): 6.17%

[1] : Kaur, Parneet, et al. "Foodx-251: a dataset for fine-grained food classification." arXiv preprint arXiv:1907.06167 (2019).

Approcci:

- Semi Supervised Learning
- Transfer Learning
- Self Supervised Learning

Semi Supervised Learning

Semi Supervised Learning

Perché?

- Il semi-supervised learning permette di sfruttare dati non etichettati
- L'ampia presenza di dati non etichettati rende l'approccio interessante al nostro caso

Sono state provate varie tecniche:

- Content-based Image Retrieval
- LabelSpread
- uso pseudolabel di ResNet per addestrare
- Metodo basato sul clustering

Content-based Image Retrieval

- Immagini $\leftrightarrow$ features neurali Alexnet, normalizzate (min-max, e con $\mu=0$, $\sigma=1$)
- calcolo matrice delle distanze (L1/L2²/L2) tra immagini di cui ho la GT
- per ogni immagine "query": ho la distanza tra lei e le immagini "indexed"
- creo lista ordinata di immagini rilevanti wrt la query, in base alla distanza
- calcolo metriche IR

I risultati sono scarsi: tabella 1.

riproviamo facendo:

- PCA con 10 dimensioni mantenute
- outlier removal: tolgo i punti lontani dal baricentro (Mahalanobis)

I risultati migliorano ma non abbastanza (tab.2)

-> metodo non messo in produzione

tab.1	Cityblock	L2
AP@5	0.0088	0.0089
AP@10	0.0078	0.0078

tab.2	Cityblock	L2
AP@5	0.0134	0.0132
AP@10	0.0109	0.0109

LabelSpread

- Immagini ↔ features neurali MobileNet (80% dati di train + 10% immagini random dall'unlabeled set)
- LabelSpreading: ottengo pseudolabels per i dati unlabeled
 - LabelSpreading costruisce un grafo delle similarità tra i dati, propaga le labels fino a convergenza
- labels+pseudolabels sono usate per finetunare Resnet50
- baseline (Resnet50 con finetuning solo su trainset): Accuracy: 21.41 %
- risultati sul validation (20% di train) non buoni:
 - accuracy: 20.12%
 - top-5 accuracy: 43.72%

classification for training

- partendo da un buon modello (Resnet50 finetunato sull'80% dei dati di train)
 - classifica le immagini unlabeled
 - prendo le immagini su cui la confidence > soglia
 - ci addestro il modello ulteriormente
 - baseline sul validation(20% trainset): 32% di accuracy
 - il nostro modello al più fa 30% di accuracy
-
- prove fatte su: soglia, learning rate del finetuning.

Metodo basato sul clustering

- Idea: Identifico 251 cluster di immagini (con e senza etichetta) ed assegno ad immagini non etichettate l'etichetta di maggioranza del cluster.
- Feature Neurali (ResNet 50 addestrata su FOOD101 + PCA)
- KMeans con K=251
 - Permette di definire un numero specifico di cluster da ricercare
 - Fornisce i centroidi, permettendo di calcolare metriche di "confidenza" come la distanza
 - I centroidi "contengono informazioni" basate anche su immagini senza label
- Addestrato solo su un sottogruppo di immagini (limitazioni di memoria)
- Estratte il 20% di immagini più vicine ad ogni centroide, assegnando la label del cluster
- Risultati valutati su classificazione

Transfer Learning

Transfer Learning

Perché?

- Per poter ottenere buoni risultati serve un approccio data-driven
- Per allenare reti profonde da zero è necessario avere:
 - Molti dati
 - Molte risorse computazionali
- Il Transfer Learning permette di sfruttare reti pre-allenate, sfruttandone le feature neurali.

Transfer Learning

Perché?

- Supponendo che le reti siano state addestrate su dominio non troppo diverso dal nostro
- Allora le feature (gli output dei vari strati) delle reti in questione dovrebbero essere utili a:
 - Riconoscere feature di “basso livello” quali bordi, contorni, nel caso di output di strati iniziali della rete
 - Mappare immagini di classi diversi in posizioni diverse nello spazio delle feature, nel caso di output di strati intermedi/finali della rete
- Sfruttando queste feature come base di partenza per dei classificatori si dovrebbero ottenere dei buoni risultati

Transfer Learning

Quali Reti?

- AlexNet - ImageNet1k
- GoogLeNet - ImageNet1k
- ResNet50 - ImageNet1k
- ResNet50 - Food101

Transfer Learning

AlexNet – ImageNet1k

- E' la prima rete su cui sono state fatti delle prove su transfer learning
- Le tecniche applicate sono state:
 - Classificatori non-neurali a feature di alto livello
 - Fine-tune degli ultimi strati fc
 - Classificatori non-neurali a feature di basso livello

Transfer Learning

AlexNet – ImageNet1k

Classificatori Non neurali,
feature di alto livello

Facendo varie prove i risultati migliori sono stati ottenuti dalle feature del penultimo strato fc.

I classificatori sono stati applicati sulle prime 500 componenti della PCA ottenuta dalle feature neurali.

I classificatori applicati sono stati i seguenti:

- Linear Support Vector: 7.57 %
- Rbf Support Vector: 11.35 %
- KNN with 1 neighbors: 5.88 %
- KNN with 10 neighbors: 6.97 %
- KNN with 20 neighbors: 5.78 %
- KNN with 50 neighbors: 7.37 %
- Naive Bayes: 7.87 %

Transfer Learning

AlexNet – ImageNet1k

Fine-tune degli ultimi strati fc

Facendo varie prove i risultati migliori sono stati con fine-tuning degli ultimi 2 strati fc.

Con i seguenti iperparametri:

Optimizer: SGD, lr = 0.001, momentum = 0.9

n° epoche: 60

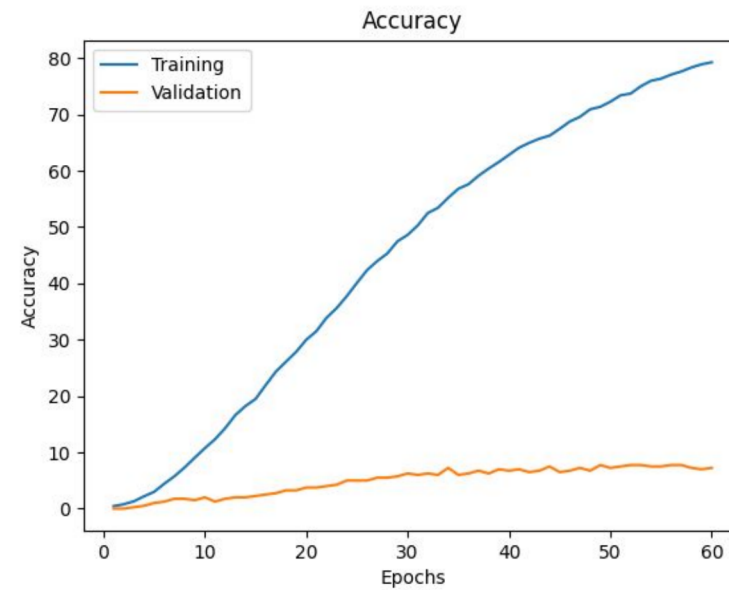
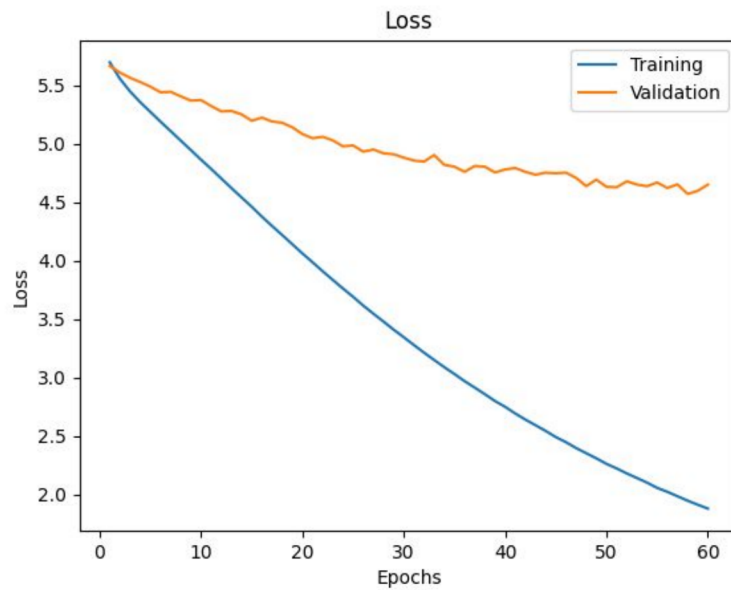
batch size: 64

Risultati:

Accuracy: 11.65 %

5-Accuracy: 26.69 %

Grafici di training del modello precedente



Transfer Learning

AlexNet – ImageNet1k

Classificatori Non neurali,
feature di basso livello

Facendo varie prove i risultati migliori sono stati ottenuti dalle feature dal terzo strato convoluzionale.

I classificatori sono stati applicati sulle prime 500 componenti della PCA ottenuta dalle feature neurali.

I classificatori applicati sono stati i seguenti:

- Linear Support Vector: 1.39 %
- Rbf Support Vector: 4.68 %
- KNN with 1 neighbors: 1.99 %
- KNN with 10 neighbors: 1.59 %
- KNN with 20 neighbors: 1.69 %
- KNN with 50 neighbors: 2.49 %
- Naive Bayes: 2.19 %

Conclusioni sui risultati di AlexNet

1. Sono più efficaci le feature di alto livello rispetto a quelle di basso livello
2. Eseguire il fine-tuning è più efficace rispetto ai classificatori non neurali. Anche se Rbf support vector e Fine-tune portano a risultati comparabili
3. Potrebbero servire delle reti più profonde per estrarre feature più discriminanti

Transfer Learning

GoogLeNet- ImageNet1k

- La prima rete profonda che abbiamo testato è stata GoogLeNet
- Le tecniche applicate sono state:
 - Classificatore neurale a feature di alto livello
 - Classificatori non-neurali a feature di alto livello

Transfer Learning

GoogLeNet– ImageNet1k

Classificatori Non neurali,
feature di alto livello

Le feature vengono estratte dallo strato precedente allo strato fully connected.

I classificatori sono stati applicati sulle prime 500 componenti della PCA ottenuta dalle feature neurali.

I classificatori applicati sono stati i seguenti:

- Linear Support Vector: 9.46 %
- Rbf Support Vector: 17.33 %
- KNN with 1 neighbors: 9.06 %
- KNN with 10 neighbors: 10.65 %
- KNN with 20 neighbors: 11.25 %
- KNN with 50 neighbors: 12.64 %
- Naive Bayes: 12.84 %

Transfer Learning

GoogLeNet- ImageNet1k

Classificatore neurale,
feature di alto livello

Facendo varie prove i risultati migliori sono stati con un classificatore neurale fc a 2 strati.

Con i seguenti iperparametri:

Optimizer: SGD, lr = 0.01, momentum = 0.9

n° epoche: 100

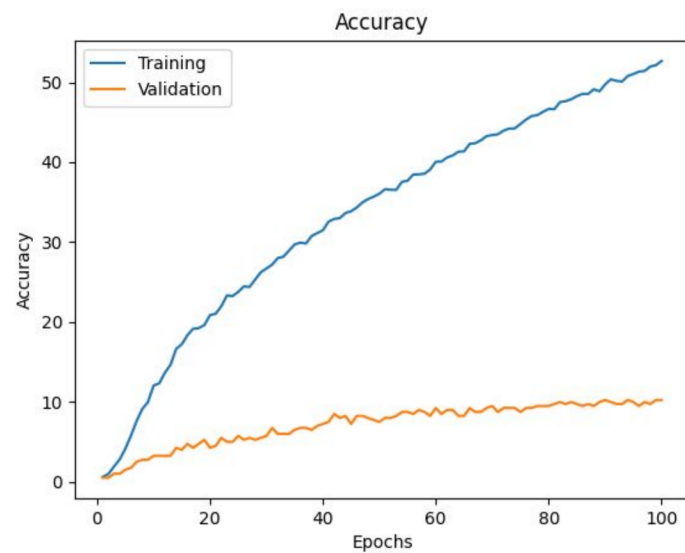
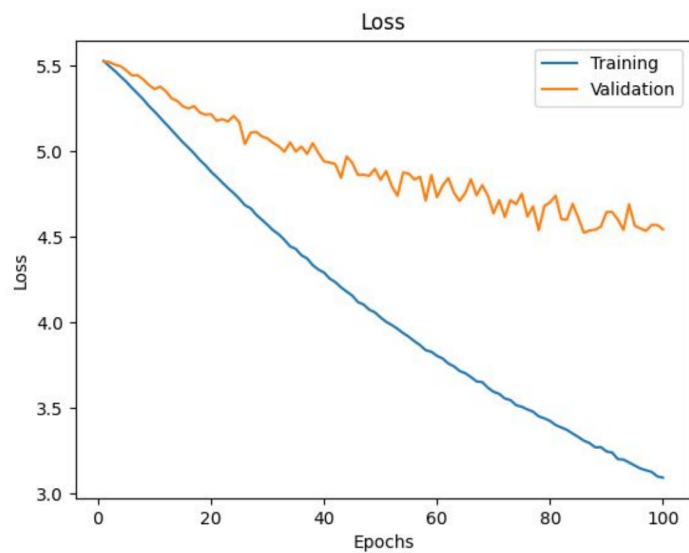
batch size: 128

Risultati:

Accuracy: 17.53 %

5-Accuracy: 36.15 %

Grafici di training del modello precedente



Conclusioni sui risultati di GoogLeNet

1. I classificatori neurali hanno performance migliori rispetto a quelli non neurali. Anche se Rbf support vector e Fine-tune portano a risultati comparabili
2. Con una rete più profonda è stato raggiunto un risultato migliore

Transfer Learning

ResNet 50– ImageNet1k

- La seconda rete profonda che è stata testata è stata ResNet50
- Le tecniche applicate sono state:
 - Classificatore neurale a feature di alto livello
 - Classificatori non-neurali a feature di alto livello

Transfer Learning

ResNet 50– ImageNet1k

Classificatori Non neurali,
feature di alto livello

Le feature vengono estratte dallo strato precedente allo strato fully connected.

I classificatori sono stati applicati sulle prime 500 componenti della PCA ottenuta dalle feature neurali.

I classificatori applicati sono stati i seguenti:

- Linear Support Vector: 11.75 %
- Rbf Support Vector: 19.42 %
- KNN with 1 neighbors: 11.06 %
- KNN with 10 neighbors: 10.26 %
- KNN with 20 neighbors: 12.25 %
- KNN with 50 neighbors: 12.35 %
- Naive Bayes: 13.44 %

Transfer Learning

ResNet 50– ImageNet1k

Classificatore neurale,
feature di alto livello

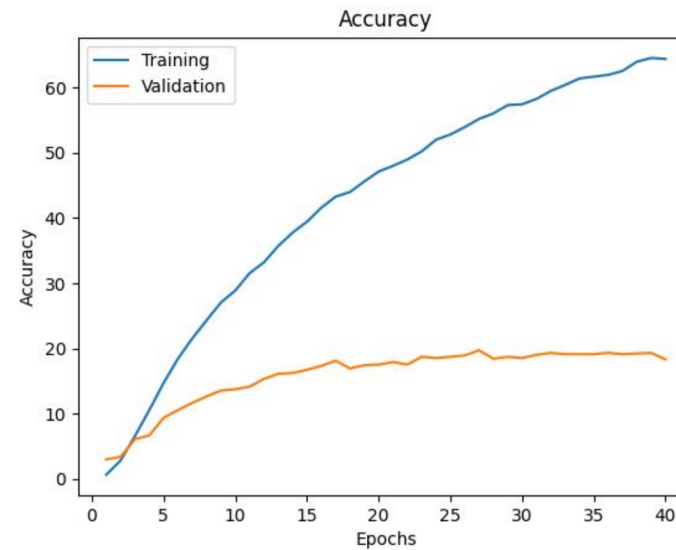
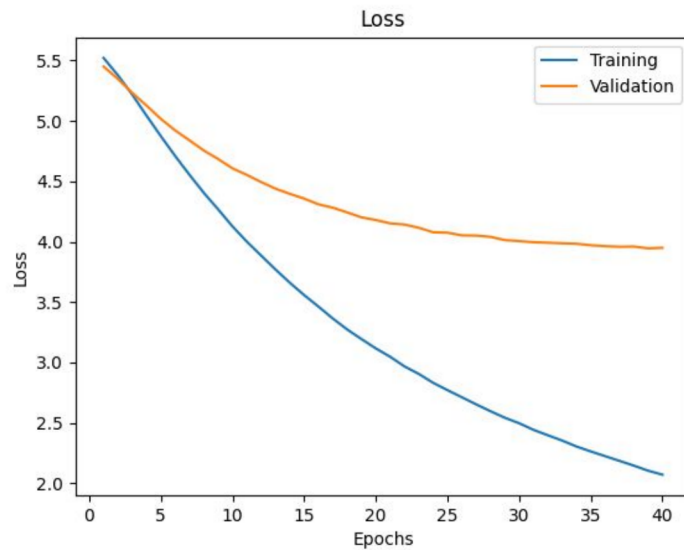
Facendo varie prove i risultati migliori sono stati con un classificatore neurale fc a 2 strati, con i seguenti parametri:

Optimizer: SGD, lr = 0.01, momentum = 0.9, n° epoche: 40, batch size: 64

Risultati:

- Accuracy: 18.32 %
- 5-Accuracy: 40.43 %

Grafici di training del modello precedente



Conclusioni sui risultati di ResNet 50

1. I classificatori neurali hanno performance migliori rispetto a quelli non neurali. Anche se Rbf support vector e Fine-tuning portano a risultati comparabili
2. Con una rete più profonda è stato raggiunto un risultato migliore
3. ResNet 50 è la rete che ha dato i risultati migliori fino ad ora

Transfer Learning

ResNet 50– Food101 [2]

- Considerati i risultati ottenuti da ResNet 50 con i pesi pre-allenati su dataset ImageNet abbiamo deciso di utilizzare la stessa metodologia su una ResNet 50 con i pesi pre-allenati su un dataset di cibo. Il dataset in questione è Food 101^[2].
- Food 101 è un dataset contenente molte immagini di cibo, di 101 classi diverse.
 - Nota: FoodX-251 include Food101^[3]
- Le tecniche applicate sono state:
 - Classificatore neurale a feature di alto livello
 - Fine-tuning dell'intera rete

[2] <https://github.com/Herick-Asmani/Food-101-classification-using-ResNet-50>

[3] Bossard, Lukas, Matthieu Guillaumin, and Luc Van Gool. "Food-101—mining discriminative components with random forests."

Transfer Learning

ResNet 50– Food101

Classificatore neurale,
feature di alto livello

Facendo varie prove i risultati migliori sono stati con un classificatore neurale fc a 2 strati, con i seguenti parametri:

Optimizer: SGD, lr = 0.01, momentum = 0.9, n° epoche: 60, batch size: 64

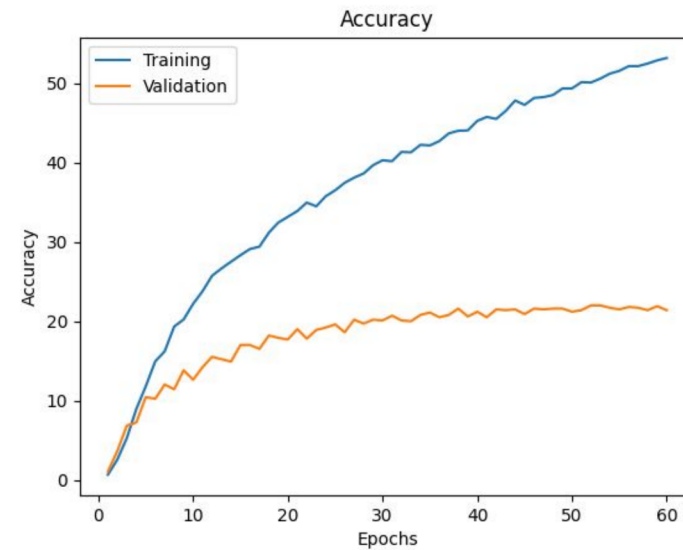
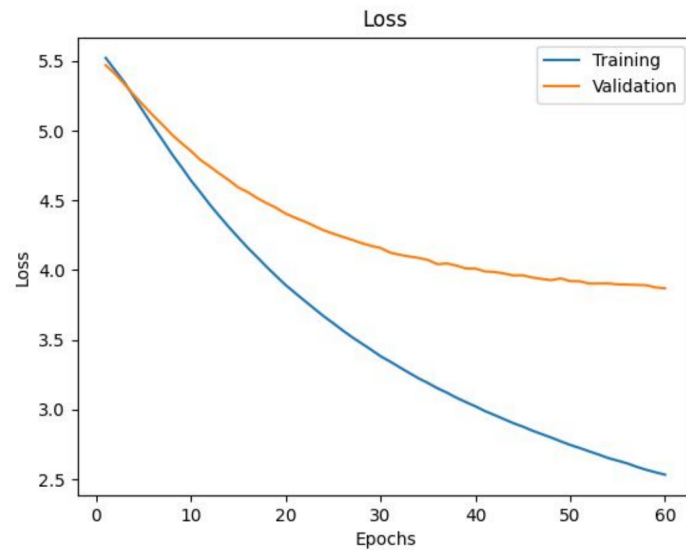
Risultati:

- Accuracy: 21.41 %
- 5-Accuracy: 44.42 %

Risultati precedenti:

- Accuracy: 18.32 %
- 5-Accuracy: 40.43 %

Grafici di training del modello precedente



Transfer Learning

ResNet 50– Food101

Fine-tune dell'intera rete

Visti i buoni risultati dati dalla classificazione di feature di alto livello abbiamo deciso di dedicare più risorse computazionali a ResNet 50 di Food 101 per vedere quanto migliorasse la rete.

Il fine tune completo della rete è stato eseguito con i seguenti parametri:

Optimizer: ADAM, lr = 0.0001

n° epoche: 10

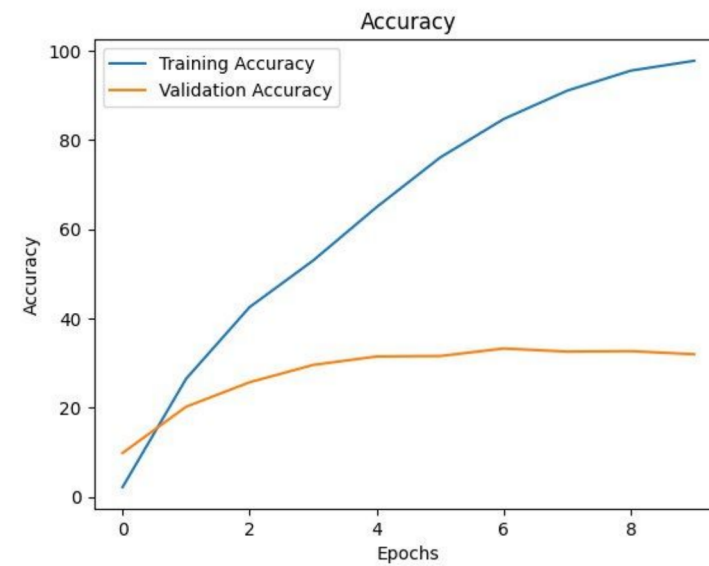
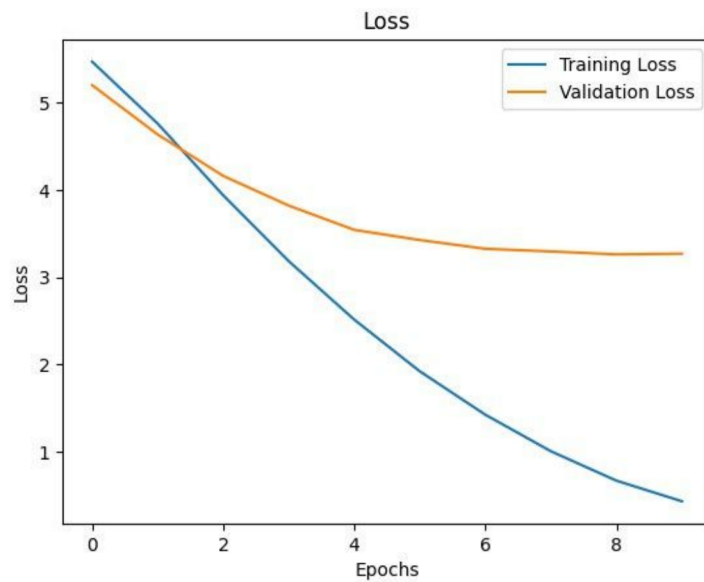
batch size: 128

Risultati:

Accuracy: 32.67 %

5-Accuracy: 59.26 %

Grafici di training del modello precedente



Transfer Learning

ResNet 50– ImageNet

Fine-tune dell'intera rete

Per determinare l'importanza dei pesi iniziali della rete nel risultato finale del training abbiamo deciso di eseguire il fine tuning della rete con i pesi di ImageNet con gli stessi iperparametri.

Il fine tune completo della rete è stato eseguito con i seguenti iperparametri:

Optimizer: ADAM, lr = 0.0001

n° epoche: 10

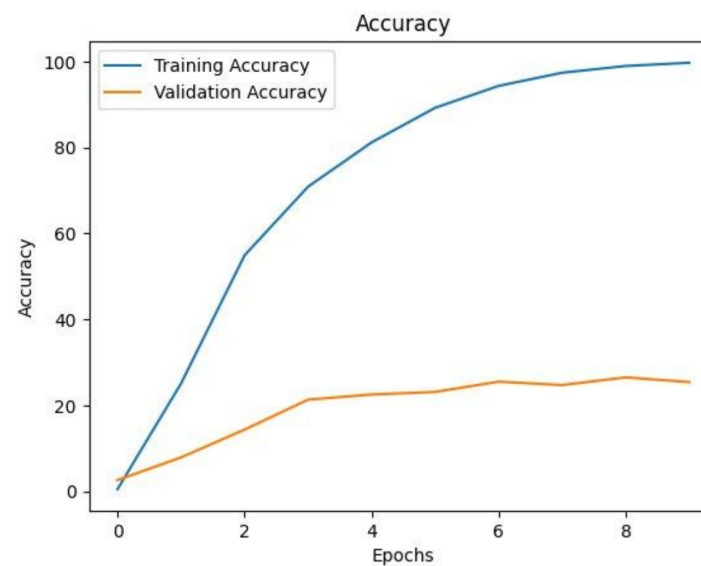
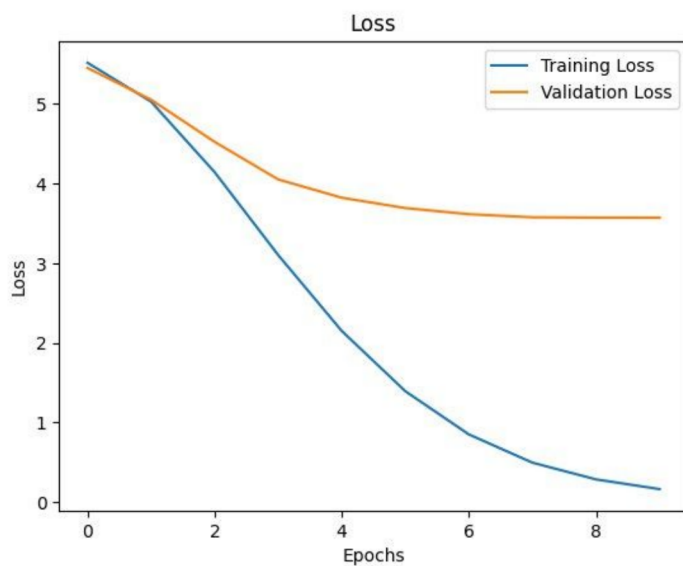
batch size: 128

Risultati:

Accuracy: 25.40 %

5-Accuracy: 49.40 %

Grafici di training del modello precedente



Conclusioni sui risultati del Transfer Learning

1. I classificatori neurali hanno performance migliori rispetto a quelli non neurali. Anche se Rbf support vector e Fine-tune portano a risultati comparabili
2. Con una rete più profonda è stato raggiunto un risultato migliore
3. ResNet 50 è la rete che ha dato i risultati migliori fino ad ora
4. Partire da pesi di ResNet 50 pre-addestrati su immagini di cibo ha portato ai risultati migliori
5. Eseguire fine-tuning sull'intera rete è computazionalmente molto pesante ma migliora molto i risultati

Self Supervised Learning

Self Supervised Learning

Perchè?

- Il self supervised learning (SSL) permette di sfruttare dati non etichettati mediante la definizione di obiettivi “intermedi”
- Questi obiettivi “intermedi” permettono di definire etichette (rilevanti per questi obiettivi intermedi) a partire dai dati non etichettati, dando la possibilità al modello di apprendere feature rilevanti per un eventuale obiettivo successivo
- L’ampia presenza di dati non etichettati rende l’approccio molto promettente

Self Supervised Learning

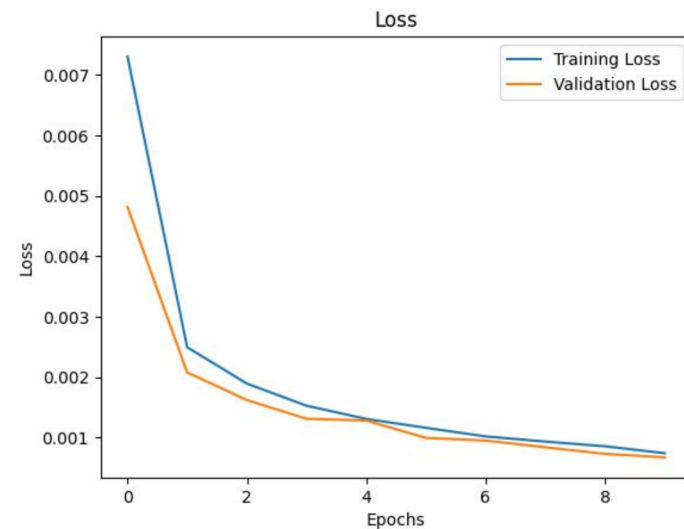
Approcci

- Ricostruzione
 - Basato su AutoEncoder
 - A partire da un'immagine in entrata, l'obiettivo della rete è ricostruire l'immagine in uscita
 - Input: immagine
 - Label: l'immagine di input
 - l'obiettivo è quello di utilizzare successivamente l'encoder per estrarre feature per la classificazione
- Colorizzazione
 - Basato su struttura Encoder - Decoder
 - A partire da un'immagine a colori, questa viene convertita nello spazio colore Lab. L'obiettivo della rete sarà quello di ricostruire i canali cromatici (ab) a partire dal canale di luminanza (L)
 - Input: canale L di un immagine a colori
 - Label: canali ab di un immagine a colori
 - L'obiettivo è quello di utilizzare successivamente l'encoder per estrarre feature per la classificazione

Self Supervised Learning

Ricostruzione

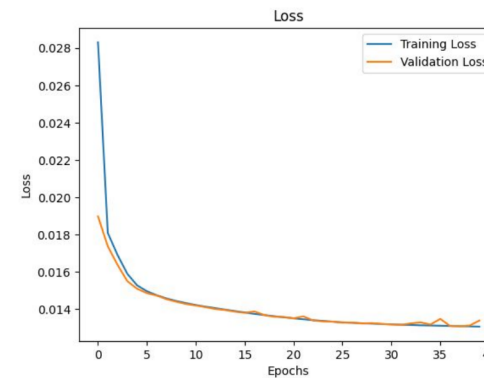
- Definito un semplice AutoEncoder convoluzionale (Encoder: convoluzione, Decoder: convoluzione trasposta)
- Addestrato su tutte le immagini di training (con e senza label)
- Loss: Mean Squared Error
- Optimizer: Adam, LR: 1e-3, Epoche: 10



Self Supervised Learning

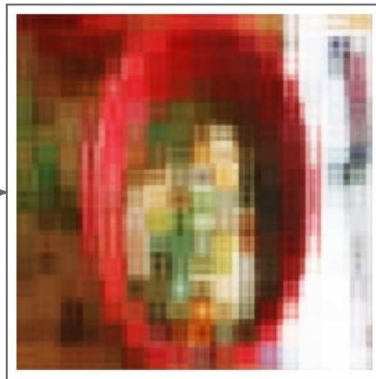
Ricostruzione

- Rischio: il modello potrebbe imparare feature univocamente utili alla ricostruzione
- Idea: sviluppo di uno Sparse Autoencoder
- Medesima architettura dell'encoder precedente, ma funzione di loss penalizzata in base alla norma L1 dello spazio latente
- Spingere il modello ad apprendere una rappresentazione latente sparsa, la quale potrebbe aiutare con la classificazione
- Loss: MSE + L1(latente), Optimizer: ADAM, LR: 1e-3, Epoche: 40

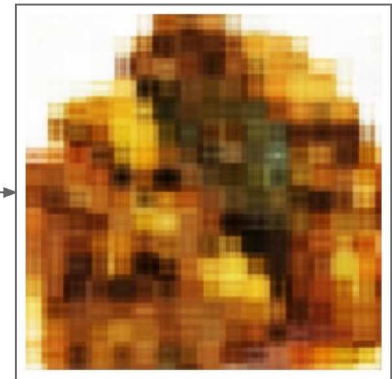


Ricostruzione:

AutoEncoder:



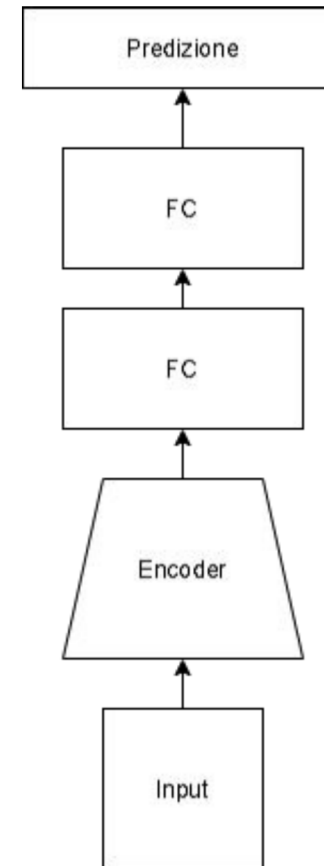
Sparse AutoEncoder:



Self Supervised Learning

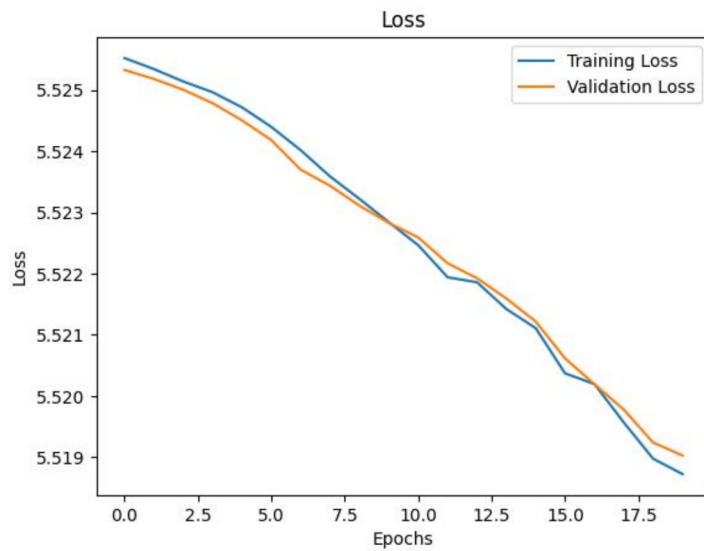
Ricostruzione

- Estratto encoder da entrambi i modelli
- Concatenati 2 layer FC dopo ogni encoder, addestrati su dati etichettati

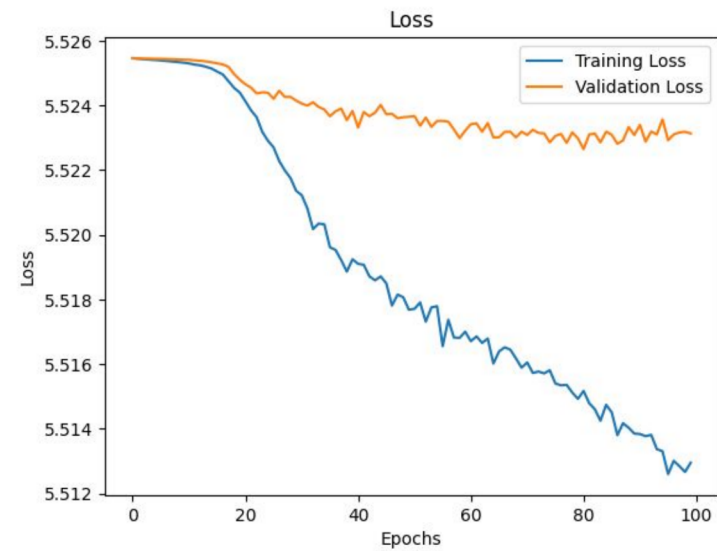


Risultati Classificazione basata su Ricostruzione

AutoEncoder:

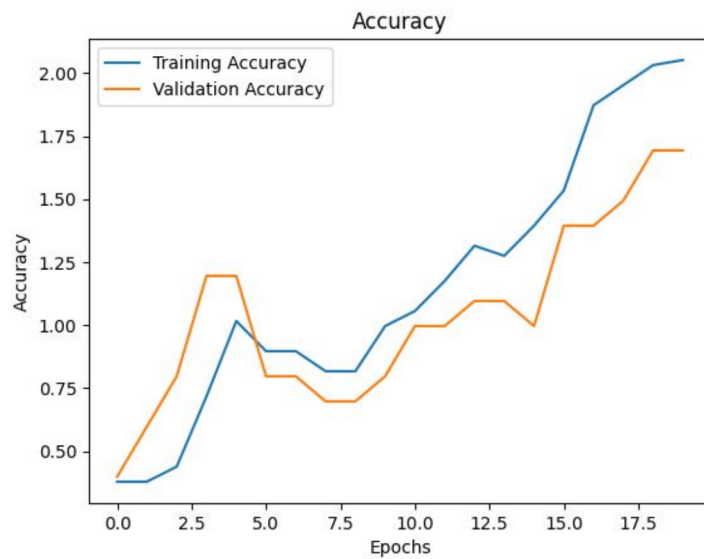


Sparse AutoEncoder:



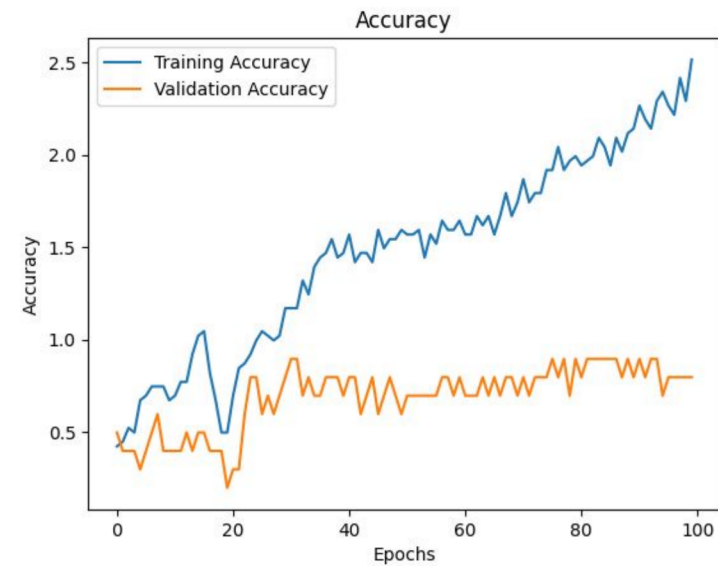
Risultati Classificazione basata su Ricostruzione

AutoEncoder:



Accuracy: 1.69%

Sparse AutoEncoder:

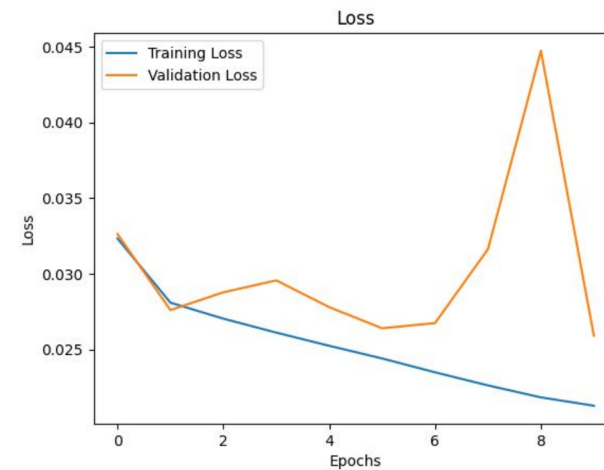


Accuracy: 0.79%

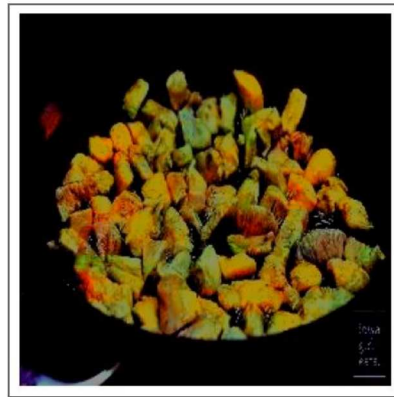
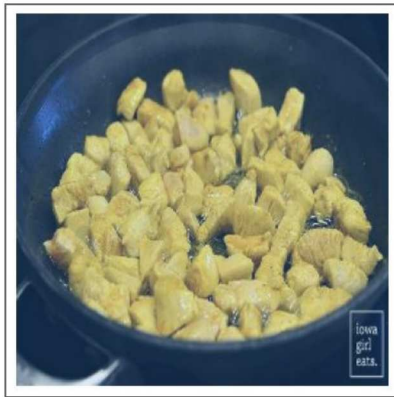
Self Supervised Learning

Colorizzazione

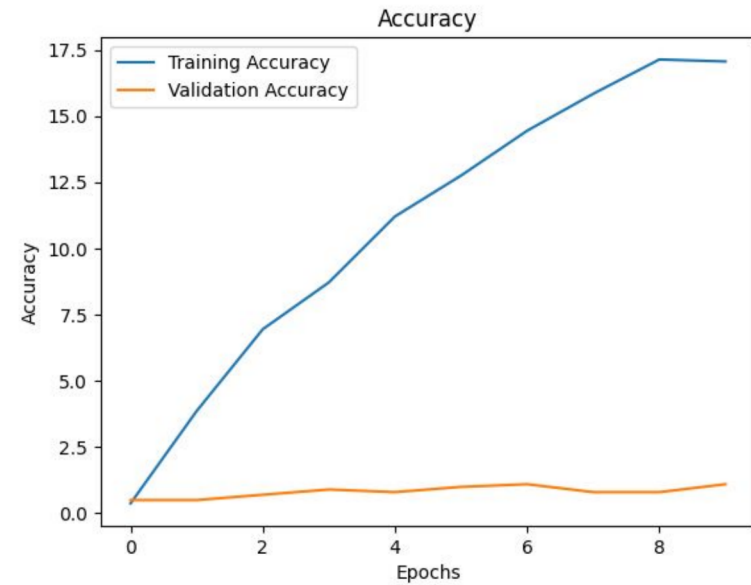
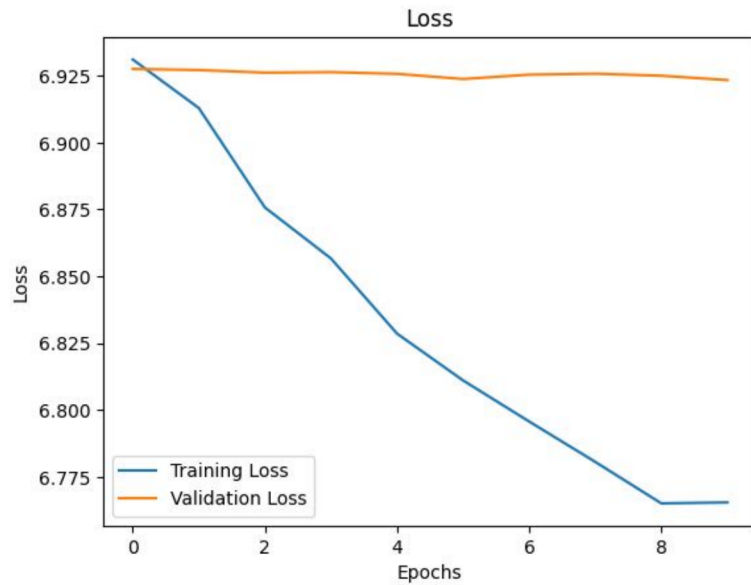
- Definita una struttura Encoder - Decoder convoluzionale
- Addestrato su tutte le immagini di training (con e senza label) nella predizione dei canali ab a partire dal canale L (spazio colore Lab)
- Loss: Mean Squared Error
- Optimizer: ADAM, LR: 1e-3, epoche: 10



Colorizzazione



Risultati Classificazione basata su Colorizzazione



Accuracy: 1.10%

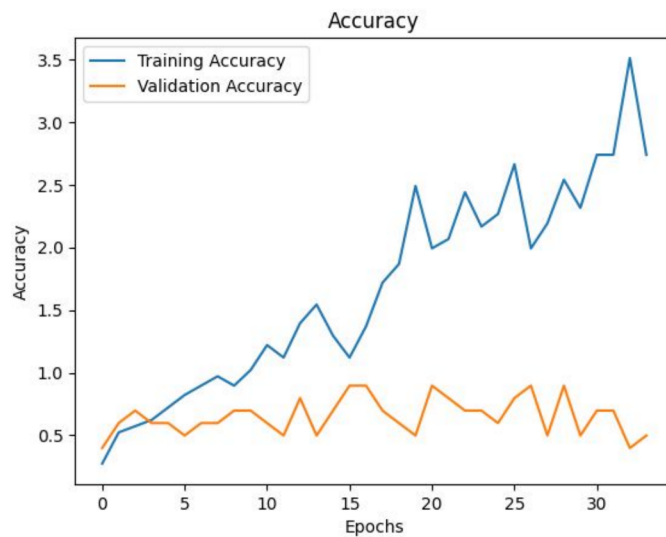
Self Supervised Learning

Problemi

- Performance pessime
- Possibili cause:
 - Problemi architetturali
 - Pochi modelli testati a causa delle poche risorse computazionali disponibili
 - Troppi pochi dati etichettati
 - Le feature estratte potrebbero essere valide, ma non abbiamo abbastanza dati per addestrare un buon classificatore

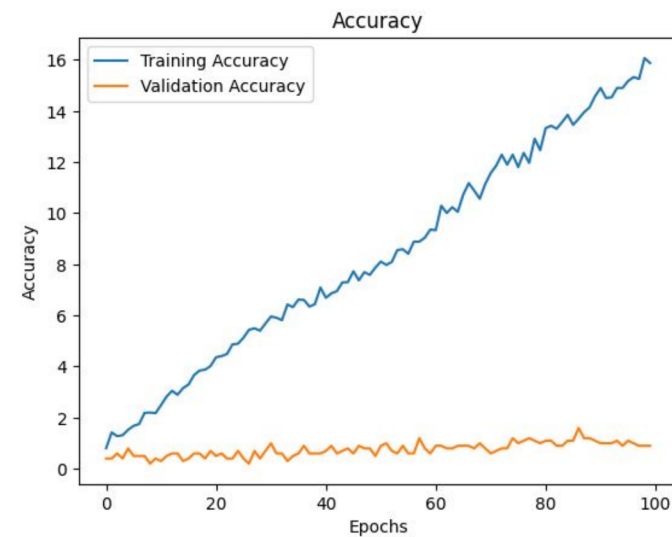
Risultati Classificazione basati su Colorizzazione

Colorizzazione + Data Augmentation:



Accuracy: 0.50%

Colorizzazione + Label Clustering:

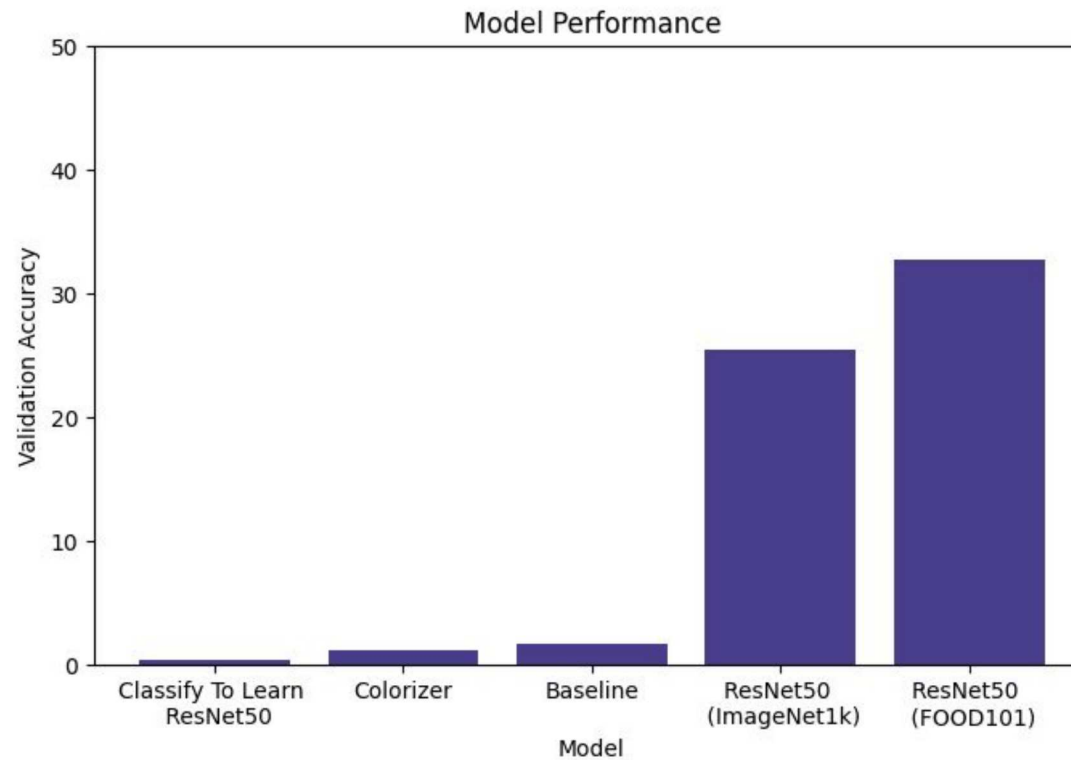


Accuracy: 0.40%

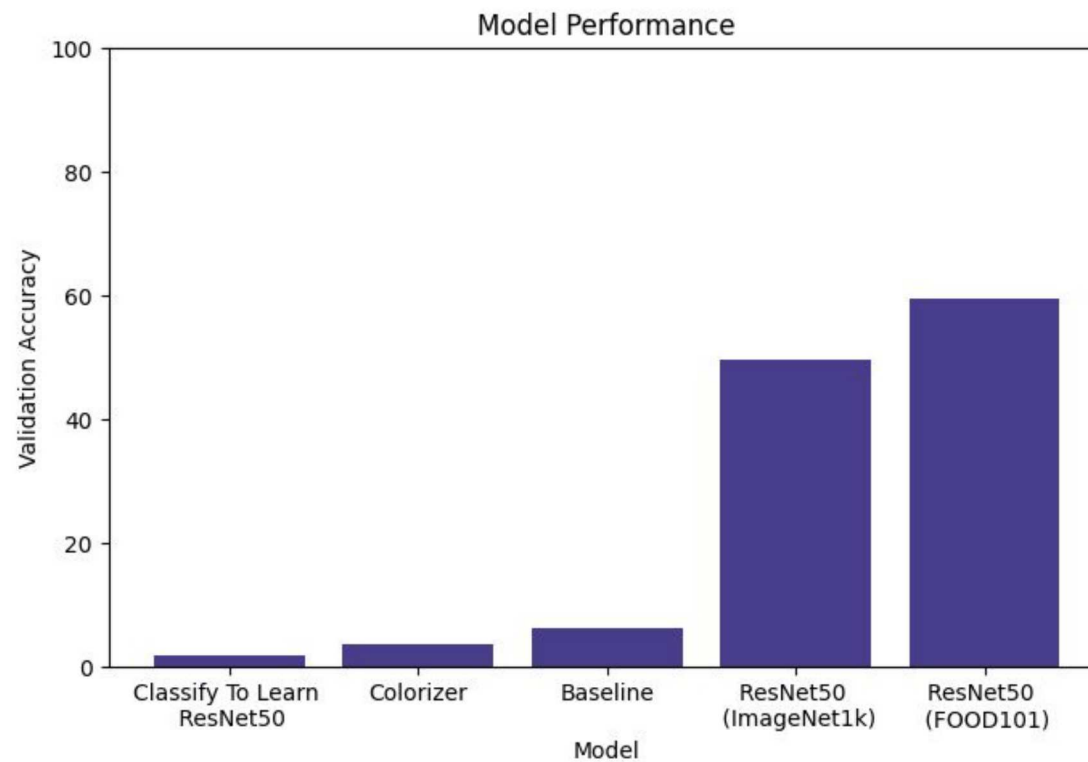
Analisi dei risultati

Validation Set

Accuracy Validation Set



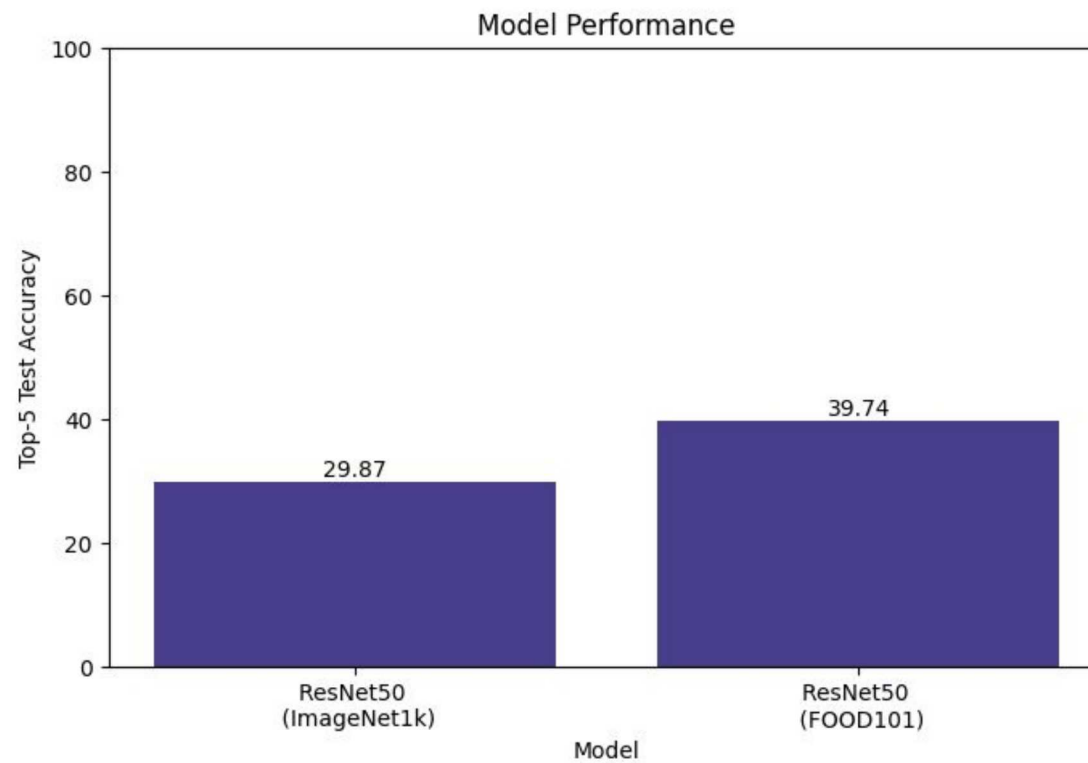
Top5 Accuracy Validation



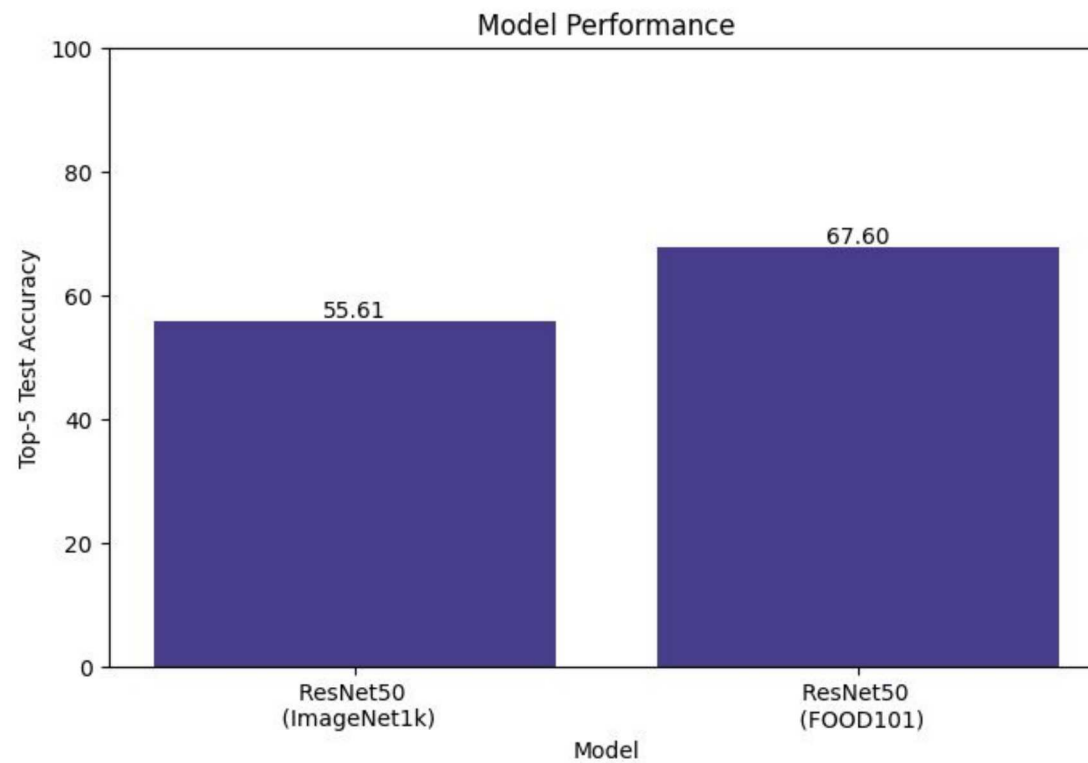
Analisi dei risultati

Test Set

Accuracy Test Set



Top5 Accuracy Test Set



Transfer Learning metrics

ImageNet1k

Accuracy: 29.87 %

5 - Accuracy: 55.61 %

Top confused Classes:

#1: ('streusel', 'crumb_cake'); has been confused 21 times

#2: ('coq_au_vin', 'beef_bourguignonne'); has been confused 20 times

#3: ('chicken_wing', 'buffalo_wing'); has been confused 19 times

#4: ('spaghetti_carbonara', 'fettuccine'); has been confused 19 times

#5: ('linguine', 'spaghetti_carbonara'); has been confused 18 times

Food101

Accuracy: 39.74 %

5 - Accuracy: 67.60 %

Top confused Classes:

#1: ('chicken_wing', 'buffalo_wing'); has been confused 26 times

#2: ('spring_roll', 'egg_roll'); has been confused 25 times

#3: ('coffee_cake', 'crumb_cake'); has been confused 25 times

#4: ('coq_au_vin', 'beef_bourguignonne'); has been confused 24 times

#5: ('baby_back_rib', 'barbecued_spareribs'); has been confused 21 times

Transfer Learning metrics

ImageNet1k

Classi Migliori:

Hamburger; 72.97% Acc (27/37)

Macaron; 72.73% Acc (40/55)

Crayfish Food; 69.57% Acc (32/46)

Classi Peggiori:

Paté; 0% Acc (0/38)

Rock Cake; 0% ACC (0/17)

Poi; 0% Acc (0/2)

Food101

Classi Migliori:

Deviled Egg; 82.76% Acc(48/58)

Red Velvet Cake; 81.82% Acc(36/44)

Beignet; 77.05% Acc(47/61)

Classi Peggiori:

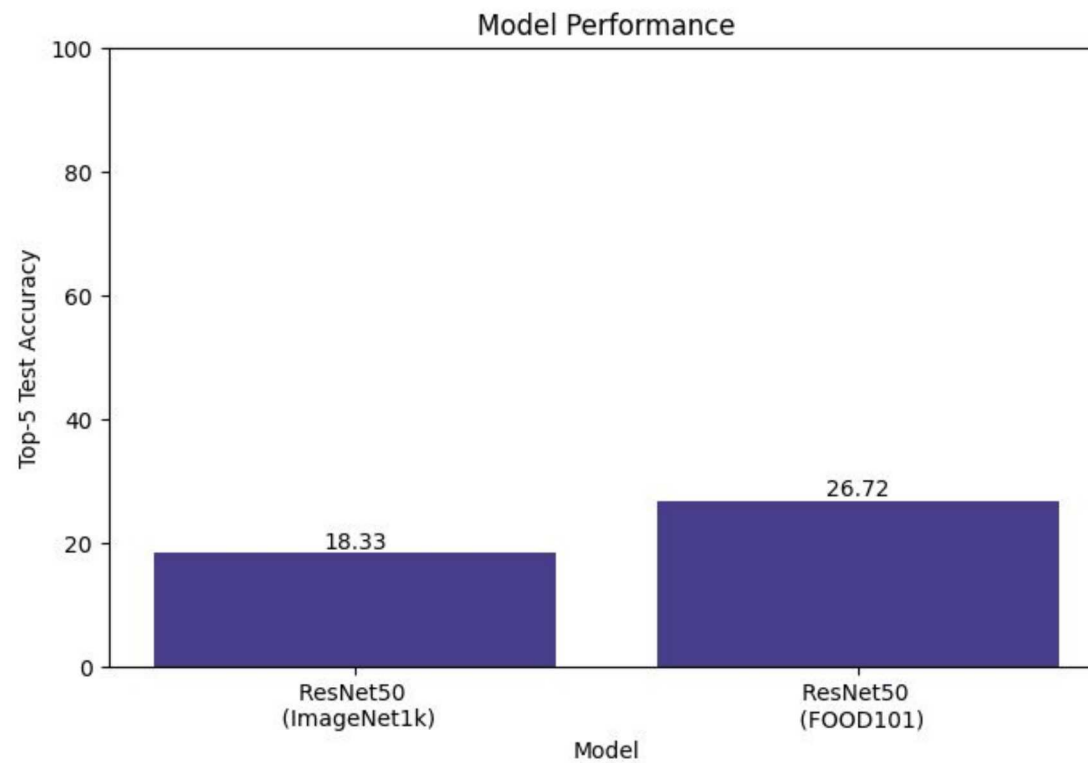
Rock Cake; 0% ACC (0/17)

Poi; 0% Acc (0/2)

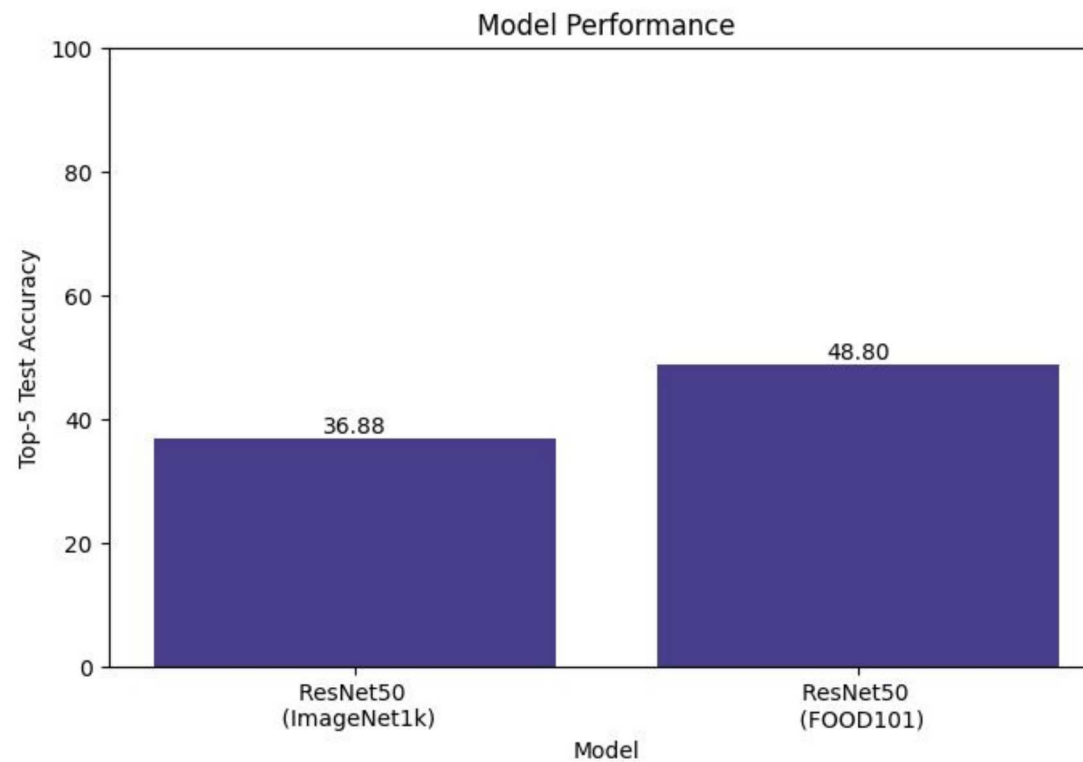
Rugulah; 0% Acc(0/2)

Ora sul Test Set Degraded...

Accuracy test Set Degraded



Top5-Accuracy test Set Degraded



Test set degraded – metriche dei modelli precedenti

ImageNet1k

Accuracy: 18.33 %

5 - Accuracy: 36.88 %

Top confused Classes:

#1: ('coffee_cake', 'crumb_cake'); has been confused 19 times

#2: ('streusel', 'crumb_cake'); has been confused 17 times

#3: ('coq_au_vin', 'beef_bourguignonne'); has been confused 15 times

#4: ('chicken_wing', 'buffalo_wing'); has been confused 14 times

#5: ('rigatoni', 'penne'); has been confused 13 times

Food101

Accuracy: 26.72 %

5 - Accuracy: 48.80 %

Top confused Classes:

#1: ('coq_au_vin', 'beef_bourguignonne'); has been confused 23 times

#2: ('streusel', 'crumb_cake'); has been confused 18 times

#3: ('chicken_wing', 'buffalo_wing'); has been confused 17 times

#4: ('beef_tartare', 'steak_tartare'); has been confused 13 times

#5: ('adobo', 'coq_au_vin'); has been confused 13 times

Test set degraded – metriche dei modelli precedenti

ImageNet1k

Classi Migliori:

Stuffed Tomato; 50% Acc (25/50)

Mussel; 50% Acc (17/34)

Scotch Egg; 48.28% Acc (28/58)

Classi Peggiori:

Fritter; 0% Acc (0/53)

Scallop; 0% Acc (0/24)

Rock Cake; 0% ACC (0/17)

Food101

Classi Migliori:

Mussel; 76.47% Acc(26/36)

Red Velvet Cake; 68.18% Acc(30/44)

Guacamole; 62.75% Acc(32/51)

Classi Peggiori:

Rock Cake; 0% ACC (0/17)

Veal Cordon Bleu; 0% Acc(0/23)

Rugulah; 0% Acc (0/11)

Data Augmentation

Data Augmentation

- Per migliorare le prestazioni su dati “sporchi”
- Definizione di una pipeline di augmentation basata sulle immagini disponibili (analisi puramente visiva)
- Operazioni (combinare casualmente):
 - Filtro di media
 - Rumore Gaussiano
 - Compressione JPEG
 - Posterizzazione
- Usato per creare un validation set degradato

Immagini test - test degraded



Immagini train - validation degraded (nostro metodo)



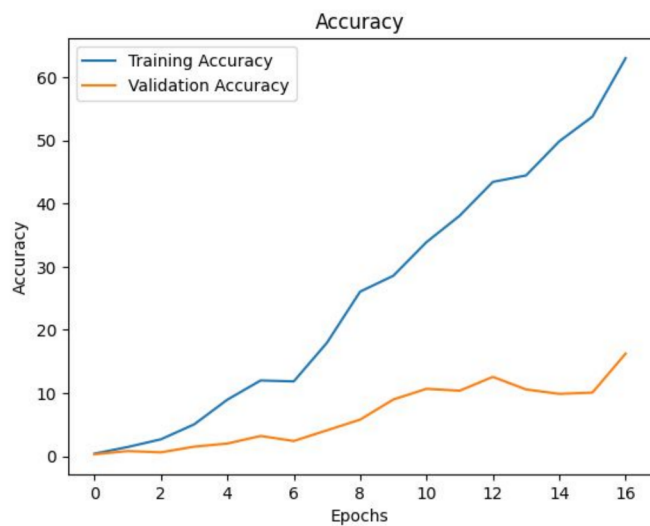
Data Augmentation

Training

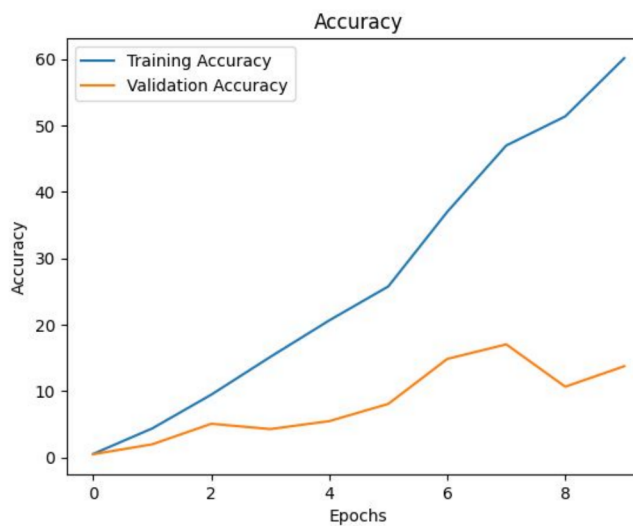
- I due modelli più promettenti della fase precedente sono stati addestrati ulteriormente utilizzando dati degradati da noi
 - Augmentation eseguita in memoria: ad ogni epoca, le stesse immagini erano degradate in maniera casuale
- Addestrati partendo sia da pesi già addestrati nello step precedente che pesi “originali”

Risultati addestramento con dati degradati: ResNet 50 (ImageNet1k)

Pesi Addestrati:

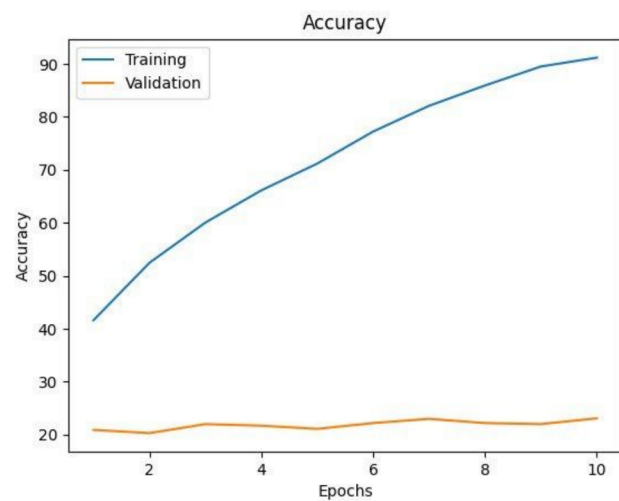


Pesi Originali:

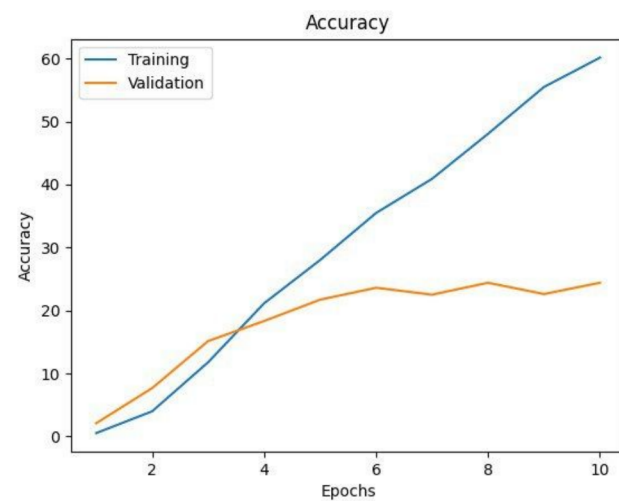


Risultati addestramento con dati degradati: ResNet 50 (FOOD101)

Pesi Addestrati:



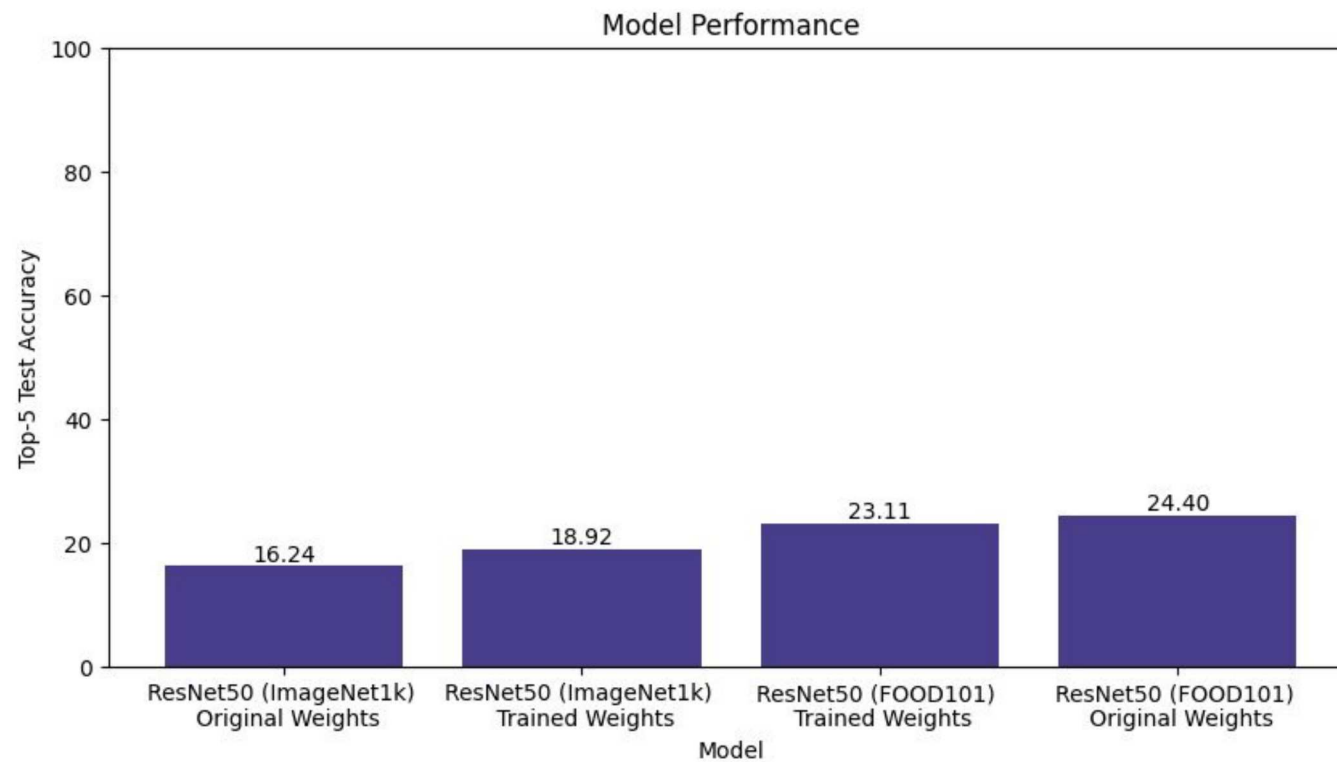
Pesi Originali



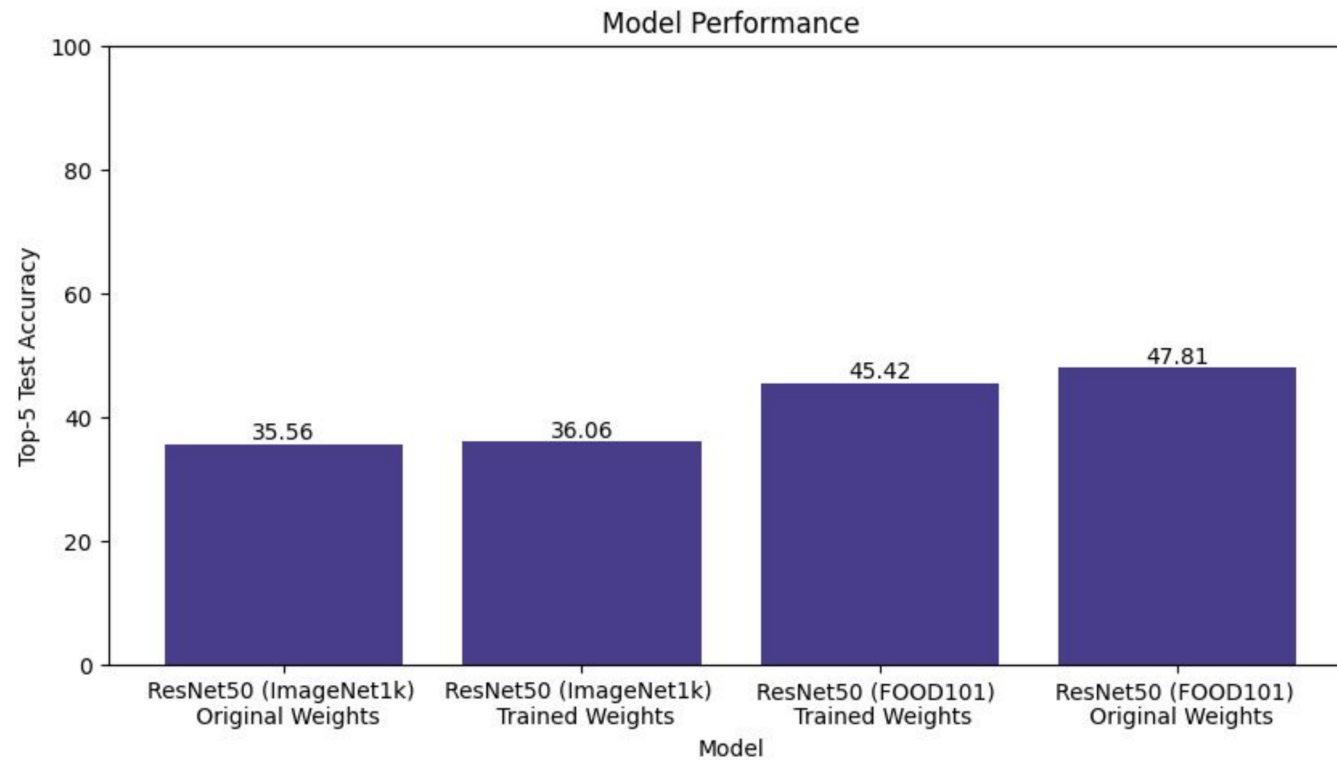
Analisi dei risultati

Validation Degraded

Accuracy Validation Degraded



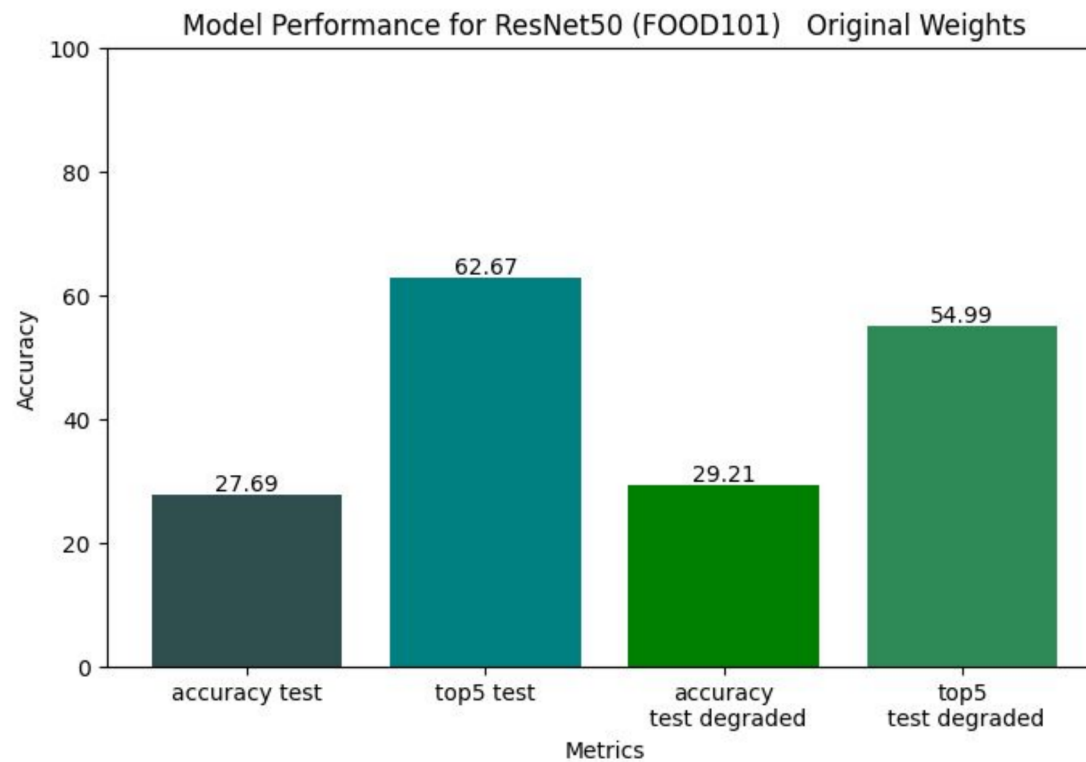
Top5 Accuracy Validation Degraded



Analisi dei risultati

Test Set / Test Set Degraded

Performance ResNet50 (FOOD101)



Test set – metriche del modello precedente

Food101

Accuracy: 27.69 %

5 - Accuracy: 62.67 %

Top confused Classes:

- #1: ('spring_roll', 'egg_roll'); has been confused 23 times
- #2: ('club_sandwich', 'bacon_lettuce_tomato_sandwich'); has been confused 22 times
- #3: ('coffee_cake', 'crumb_cake'); has been confused 22 times
- #4: ('baby_back_rib', 'barbecued_spareribs'); has been confused 22 times
- #5: ('linguine', 'spaghetti_carbonara'); has been confused 21 times

Food101

Classi Migliori:

Red Velvet Cake; 88.63% Acc(39/44)

Eggs Benedict; 86.15% Acc(56/65)

Mussel; 85.29% Acc(29/34)

Classi Peggiori:

Knish; 0% ACC (0/48)

Wonton; 0% Acc (0/48)

Syllabub; 0% Acc(0/48)

Test set degraded – metriche del modello precedente

Food101

Accuracy: 29.21 %

5 - Accuracy: 54.99 %

Top confused Classes:

#1: ('spring_roll', 'egg_roll'); has been confused 21 times

#2: ('coffee_cake', 'crumb_cake'); has been confused 21 times

#3: ('linguine', 'spaghetti_carbonara'); has been confused 19 times

#4: ('pulled_pork_sandwich', 'sloppy_joe'); has been confused 17 times

#5: ('club_sandwich', 'bacon_lettuce_tomato_sandwich'); has been confused 16 times

Food101

Classi Migliori:

Red Velvet Cake; 81.82% Acc(36/44)

Eggs Benedict; 80% Acc(52/65)

Mussel; 77.05% Acc(26/34)

Classi Peggiori:

Wonton; 0% Acc (0/48)

Confit; 0% ACC (0/37)

Galatie; 0% Acc(0/36)

Conclusioni e Sviluppi futuri

Conclusioni

- I metodi pensati per sfruttare i dati senza etichetta hanno portato a scarsi risultati, al contrario di quanto pensato all'inizio del progetto
- Invece l'allenamento di classificatori neurali ha portato buoni risultati grazie all'utilizzo di transfer learning, in particolare sfruttando pesi ottenuti da una ResNet allenata su immagini di cibo
- Applicando un'adeguata data augmentation ai dati di training e sfruttando il metodo migliore ottenuto dal transfer learning è stato possibile ottenere dei risultati paragonabili tra dataset sporco e normale

Sviluppi futuri

- Provare altre pipeline di augmentation
- pipeline di restoration (no-referece)
- Provare altri metodi di CBIR (features, distanze)
- prefiltering delle immagini unlabeled e usare solo quelle per aumentare il trainset, sia per il CBIR sia che per LabelSpread
- Sfruttare metodi Ensemble

Transfer Learning con metodo Ensemble

idea per un possibile sviluppo futuro

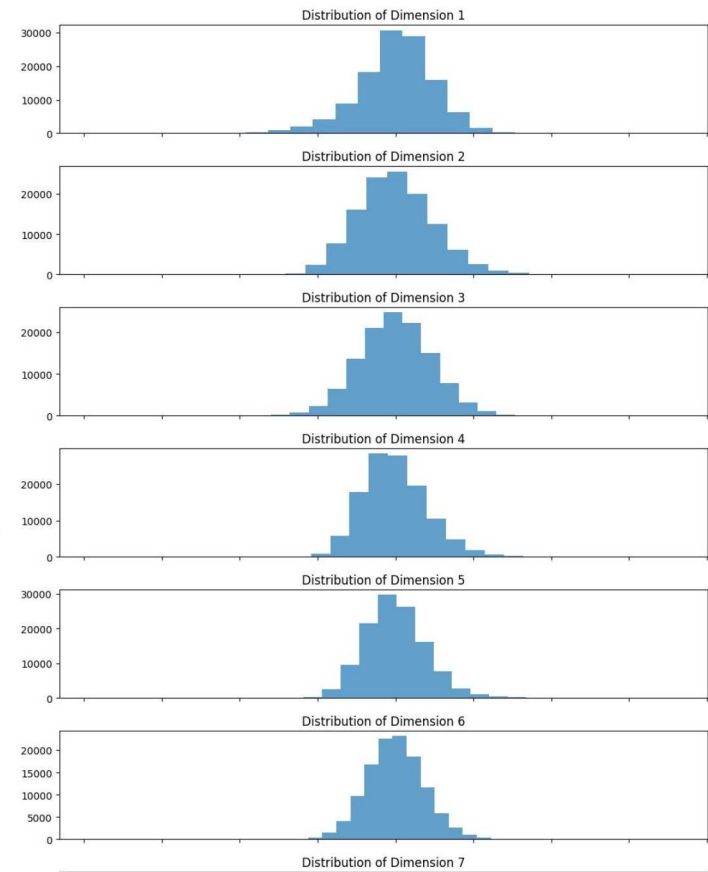
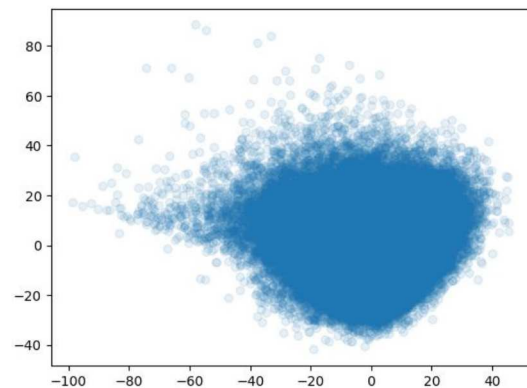
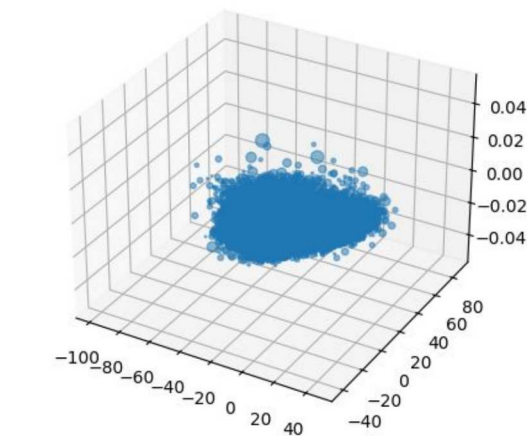
- Sono presenti molte classi, in proporzione ci sono pochissime immagini di esempio per classe
- Sono presenti molte classi che sono molto simili, quindi si potrebbero creare dei macro-gruppi di classi
- Creando dei macro-gruppi sul quale allenare un modello per gruppo si potrebbero ottenere modelli specializzati nel distinguere classi simili
- Riducendo il numero di classi per modello dovrebbe migliorare anche la capacità dei modelli di apprendere

Grazie per l'attenzione :)

Slide di supporto

CBIR

- Per il calcolo baricentro e rimozione outlier i dati sono le immagini di training+unlabeled
- la distanza usata è Mahalanobis
- peso di una dimensione $\rightarrow$ inverso della sua standard deviation



LabelSpread

- input: dati annotati e non
- costruzione grafo (i.e. matrice) delle similarità dei punti unlabeled a quelli labeled (immagine $\leftrightarrow$ nodo; distanza $\leftrightarrow$ edge)
- il peso della distanza/similarità lo si può calcolare con knn
- propagazione labels ai soli dati unlabeled:
 - la label che ottiene un dato (unlabeled) è influenzata dai dati labeled, in modo $\propto$ alla similarità
- si propaga fino a convergenza, ovvero quando le labels non cambiano più (o non abbastanza rispetto a prima).

CBIR

IR Evaluation – Precision@K (Rank-Based)

Select a rank threshold K

Compute % relevant in top K

Ignores documents ranked lower than K

Prec@3 of 2/3

Prec@4 of 2/4

Prec@5 of 3/5



Similarly for the **Recall@K**, as we know the total relevant documents per query.

Pre-processing effettuato:

Vengono applicate soltanto 2 operazioni di Pre-Processing:

1. Normalizzazione dell'immagine (porto media e varianza di ogni canale a degli specifici valori)
2. Resize dell'immagine in base alle esigenze di dimensione dell'input del modello specifico (227x227 per alexnet, 224x224 le altre reti)

RBF (Radial Basis Function) Kernel

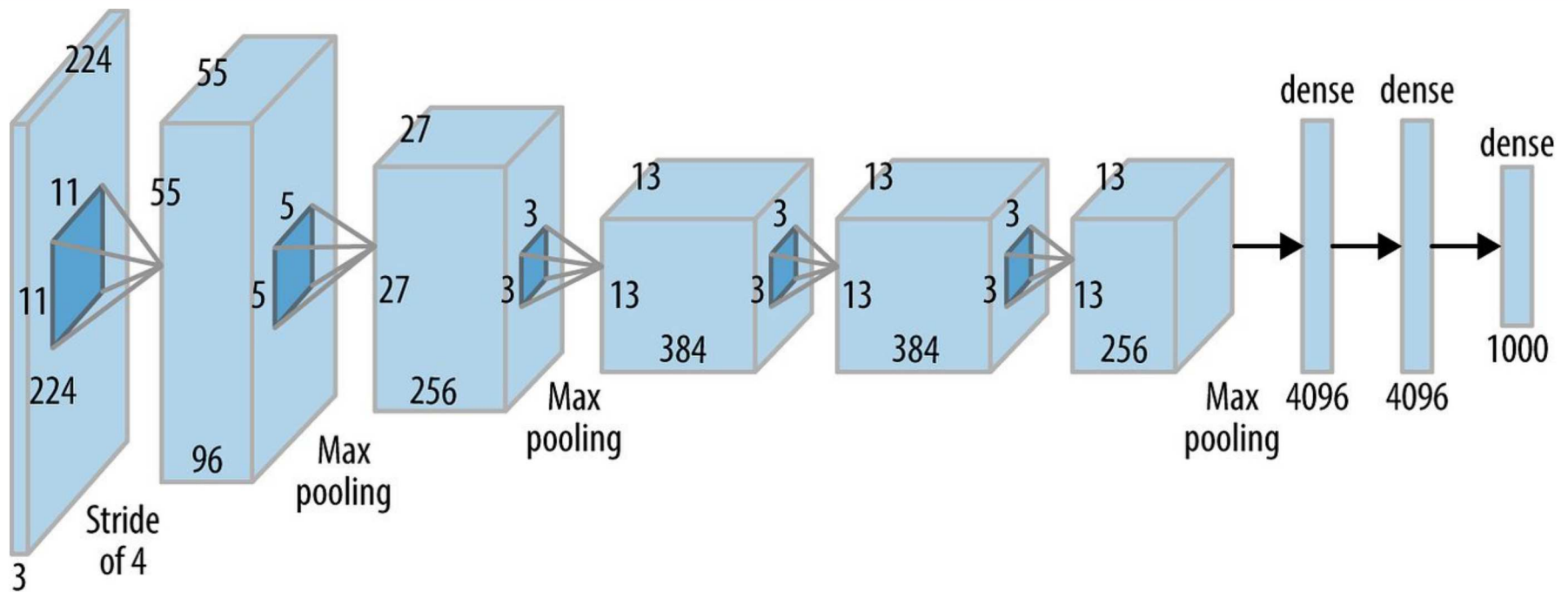
The RBF kernel on two samples $\mathbf{x} \in \mathbb{R}^k$ and $\mathbf{x}'$, represented as feature vectors in some *input space*, is defined as^[2]

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

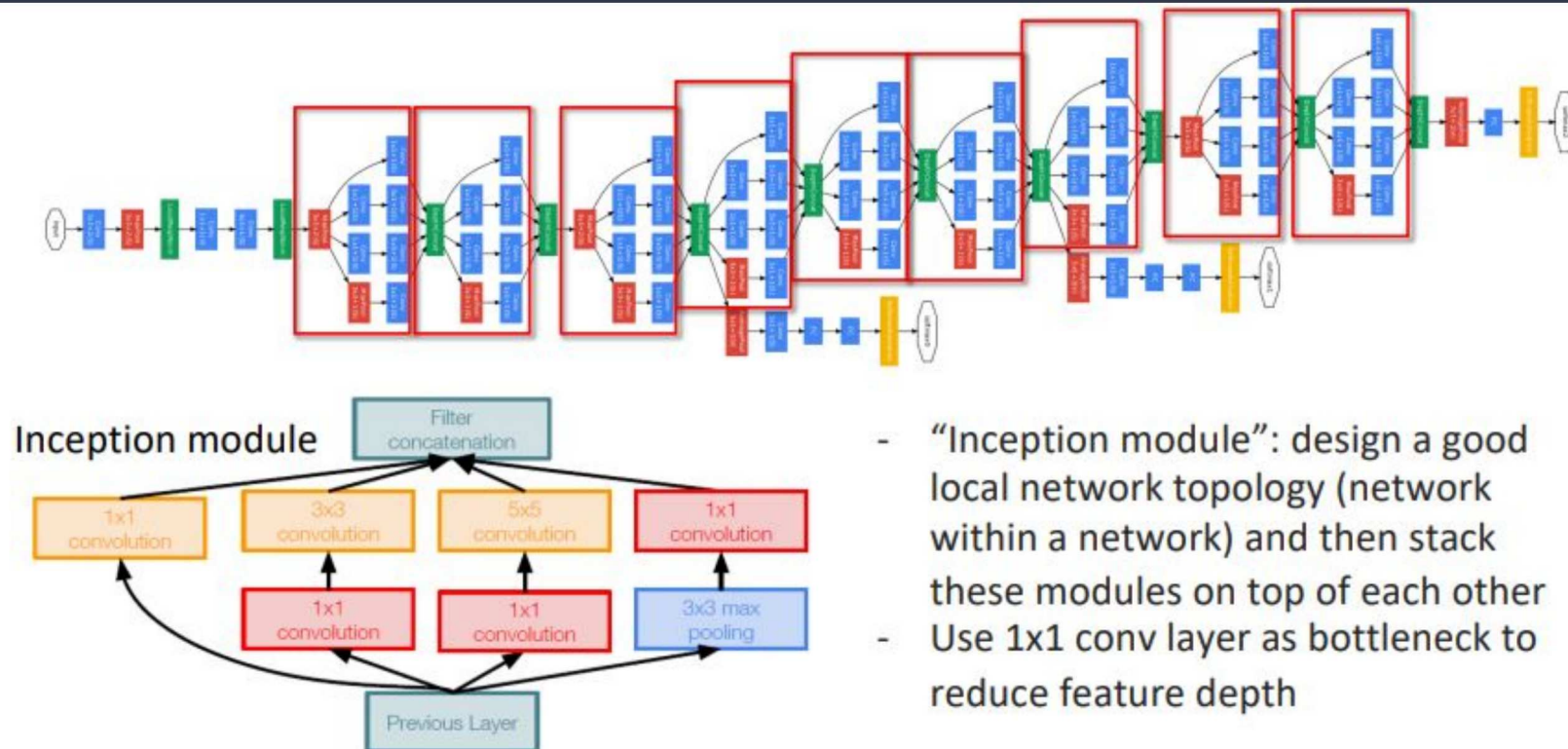
$\|\mathbf{x} - \mathbf{x}'\|^2$ may be recognized as the [squared Euclidean distance](#) between the two feature vectors. σ is a free parameter. An equivalent definition involves a parameter $\gamma = \frac{1}{2\sigma^2}$:

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$$

AlexNet

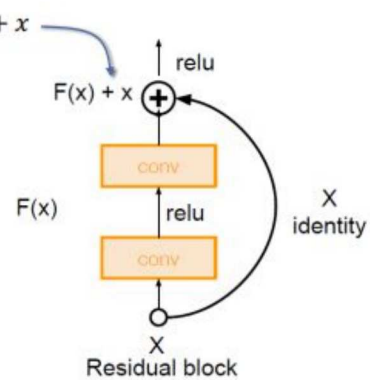
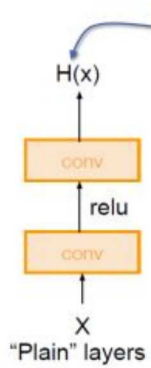


GoogLeNet

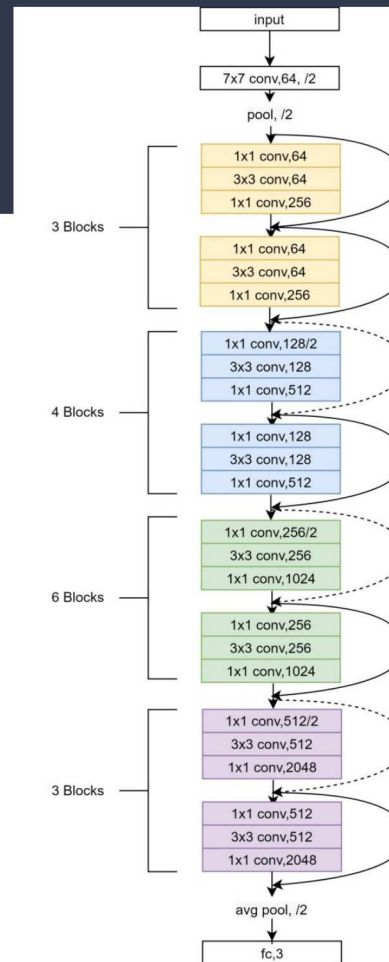


- “Inception module”: design a good local network topology (network within a network) and then stack these modules on top of each other
- Use 1x1 conv layer as bottleneck to reduce feature depth

ResNet50



Use layers to fit the residual $F(x) = H(x) - x$ instead of $H(x)$ itself



Transfer Learning con metodo Ensemble

Definizione dei Macro-Gruppi/Risultati

Gruppo 1: Uova, Pesce, Frutti di mare

Gruppo 2: Pasta, Riso, Cereali

Gruppo 3: Torte

Gruppo 4: Dolci

Gruppo 5: Panini, Tacos

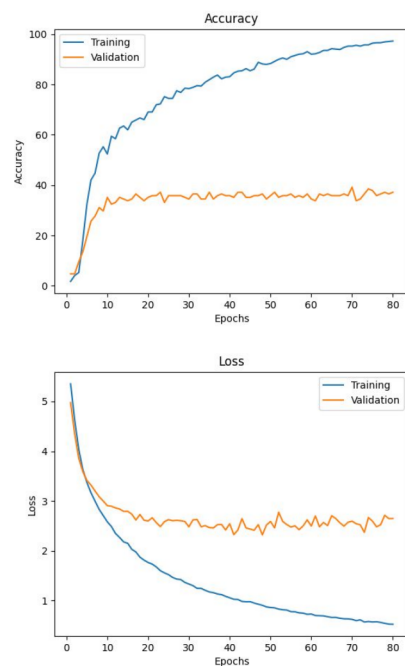
Gruppo 6: Altro

Risultati

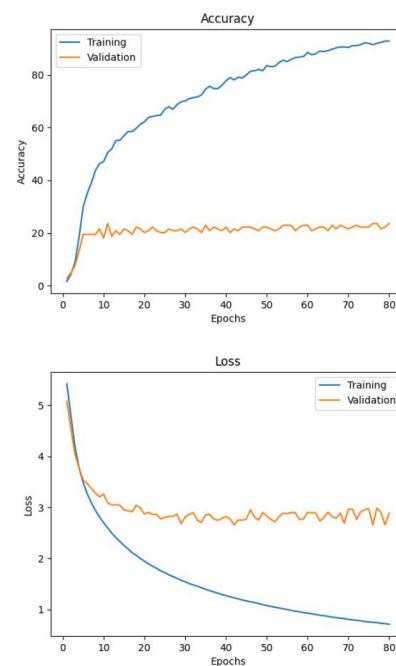
Accuracy di circa il 13%

Risultati dell'allenamento dei modelli dell'ensemble (6 modelli fc addestrati su feature di ResNet50 – imageNet1k)

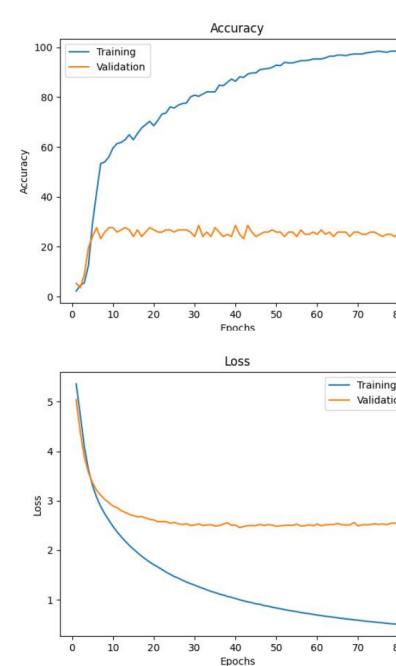
Uova, Pesce, Frutti di mare



Pasta, Riso, Cereali

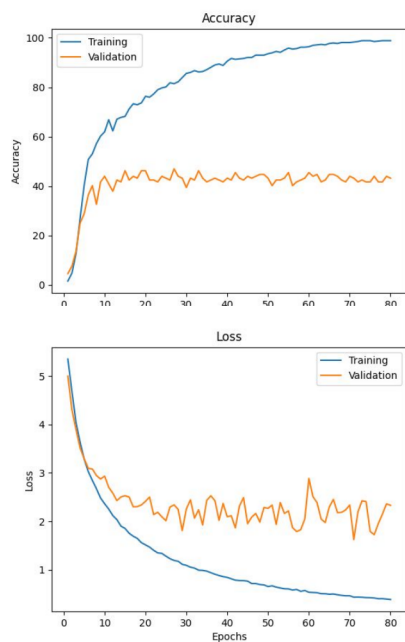


Torte

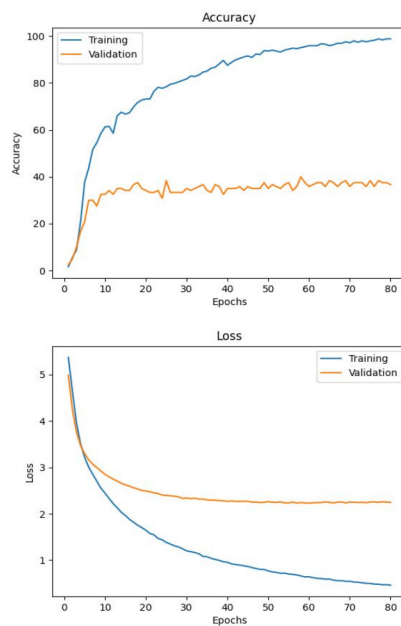


Risultati dell'allenamento dei modelli dell'ensemble (imageNet1k)

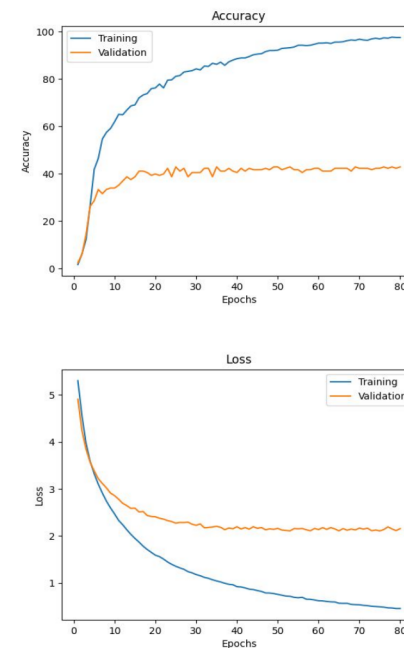
Dolci



Panini, Tacos



Altro



100

100

Transfer Learning con metodo Ensemble

Unione modelli Ensemble

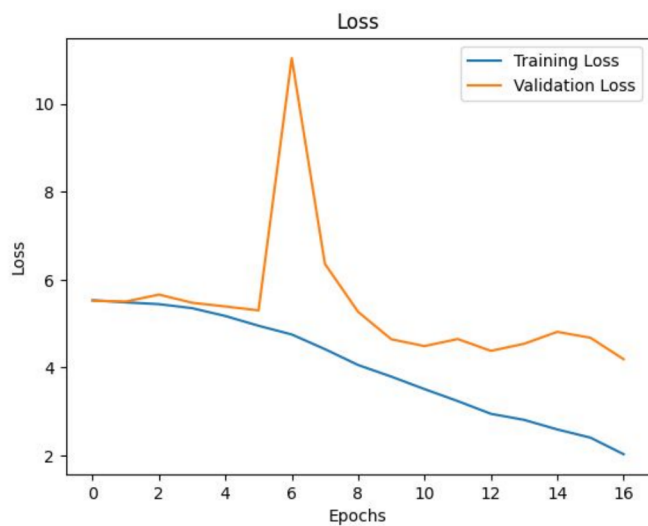
- Decisione presa prendendo come classe il massimo logit tra tutti i modelli
- Decisione presa prendendo come classe il valore massimo dopo aver applicato una softmax ai logit
- Entrambi i metodi hanno dato risultati con accuracy del circa 13%

Idee per migliorare i risultati

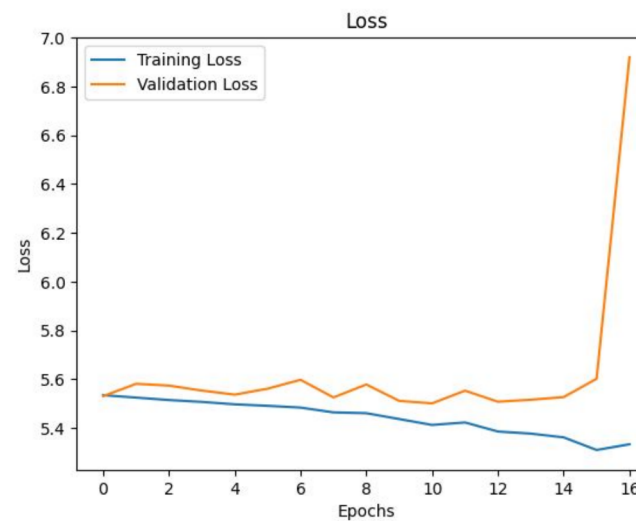
- Un modo da provare sarebbe prendere decisione in base ad un sistema di voti dei modelli
- Si potrebbe provare a separare il macro gruppo "Altro" in gruppi coerenti
- Si potrebbe provare a unire dei gruppi, forse ci sono troppi modelli nell'ensemble
- Fare in modo che ogni rete abbia un numero di neuroni finali di output pari al numero di classi del sotto-insieme del dataset che deve imparare

Risultati addestramento con dati degradati: ResNet 50 (ImageNet1k)

Pesi Addestrati:

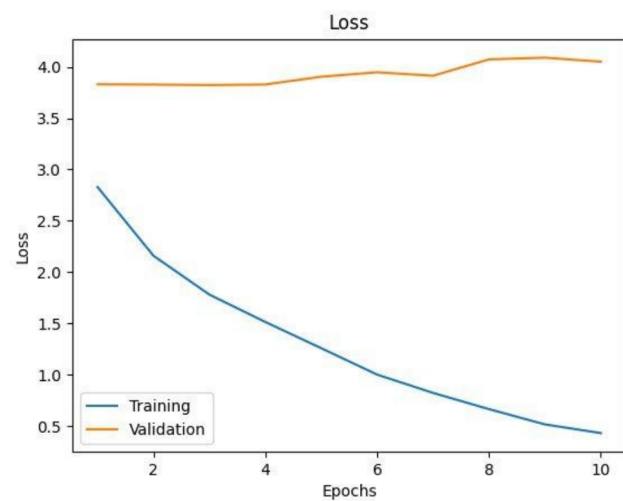


Pesi Originali:



Risultati addestramento con dati degradati: ResNet 50 (FOOD101)

Pesi Addestrati:



Pesi Originali

